

《借灯》

一盏灯，照见一段被掩埋的真相

项目文档

Final

课程名称 网站开发小学期

指导教师 赵丰年

项目组 白灯客

小组成员 罗晨菲、杨梦、于知让、高冰轩、卢正松

2026 年 9 月

目录

- 一、 项目概述 6**
 - 1.1 游戏简介 6
 - 1.1.1 概述 6
 - 1.1.2 核心玩法循环 7
 - 1.1.3 创作背景与理念 7
 - 1.2 体验定位 8
 - 1.2.1 玩家画像 8
 - 1.2.2 沉浸式体验 8
 - 1.2.3 感受主旨 9
 - 1.3 项目目标分期与实现边界 9
 - 1.3.1 分期实现 9
 - 1.3.2 实现难点与挑战 10
- 二、 游戏设计详解 12**
 - 2.1 主线剧情 12
 - 2.1.1 开场与物品 12
 - 2.1.2 三条身份线 12
 - 2.1.3 五次揭示顺序 12
 - 2.1.4 五结局设计 13
 - 2.2 探索场景 14
 - 2.2.1 场景即界面 14
 - 2.2.2 场景清单 14
 - 2.2.3 人物对话 15
 - 2.2.4 背包与线索 15
 - 2.3 核心交互 16
 - 2.3.1 剧情对话 16
 - 2.3.2 四种小游戏 16
 - 2.3.3 恐怖表现与设计 16
 - 2.3.4 存档与进度恢复 17
 - 2.4 附加功能 17
 - 2.4.1 逃离数据机房与生命值系统 17

2.4.2	成就系统	18
2.4.3	项目组信息	19
2.4.4	音效与美术	20
三、	美学与用户体验	21
3.1	视觉设计规范	21
3.1.1	整体风格	21
3.1.2	配色	21
3.1.3	字体	21
3.1.4	人物立绘	22
3.1.5	美术原则	23
3.1.6	动效规范	23
3.2	文字风格	24
3.2.1	整体调性	24
3.2.2	用语	24
3.2.3	氛围营造	24
3.3	界面设计	25
3.3.1	站点地图	25
3.3.2	线框图	25
3.3.3	设计图	25
3.4	恐怖美学设计	26
3.4.1	整体特点	26
3.4.2	音效	26
3.4.3	视频	27
3.4.4	转场	27
四、	技术方案与质量保证	29
4.1	开发与运行平台	29
4.2	技术选型	29
4.2.1	原生技术栈	29
4.2.2	浏览器本地存储	29
4.3	技术架构	30
4.4	代码组织	31
4.5	质量保证	32
4.5.1	异常与降级	32
4.5.2	测试	33

4.5.3 协作流程与 subPR 分工	33
五、 项目管理与进度	34
5.1 项目提出与开发方式	34
5.2 协作流程	34
5.2.1 subPR 的运作规则	34
5.2.2 五日迭代节奏	35
5.3 轮值角色与分工	35
5.4 迭代进度	37
5.5 流程效果	37
六、 测试与回帖情况总结	39
6.1 浏览器兼容测试	39
6.2 操作系统兼容测试	39
6.3 分辨率兼容测试	39
6.4 回帖情况汇总	40
七、 最终显示效果	43
7.1 进入与开场	43
7.2 剧情与探索	43
7.3 玩法与终局	44
7.4 系统与视听	46
八、 小组总结	49
8.1 项目开发全景复盘	49
8.1.1 V1: 链路打通 (9.3–9.8)	49
8.1.2 V2: 模块边界成文 (9.7–9.11)	49
8.1.3 V3: 接口冻结 (9.9–9.15)	50
8.1.4 V4: 按交付物分工 (9.16–9.20)	50
8.2 技术难点与解决	50
8.2.1 状态记录的信任边界	50
8.2.2 界面切换规则的合并冲突	51
8.2.3 测试通过与正式可玩的差距	51
8.2.4 存档迁移的处理原则	52
8.3 协作机制与认知	52
8.3.1 模块边界的书面化	52
8.3.2 冲突处理的组合原则	52

8.3.3	问题的归属与回退	53
8.3.4	文档冲突的处理	53
8.4	项目开发认知	54
8.4.1	可玩性问题的处理	54
8.4.2	可维护性问题的处理	54
8.4.3	验收标准的建立	54
8.5	总结与不足	55
九	个人总结	56
9.1	罗晨菲	56
9.1.1	V1: 全局、导航与存档	56
9.1.2	V2: 需求文档 v2 的制定	57
9.1.3	V3: 备战数学建模	57
9.1.4	V4: 整体完善与成品视频	57
9.1.5	回看这三周	58
9.2	杨梦	58
9.2.1	起步: 从组队到能独立提交 PR	58
9.2.2	美术创作: 画面要服务叙事	59
9.2.3	分工之外的工作	59
9.2.4	文档、日志与课堂汇报	60
9.2.5	不足与回望	60
9.3	于知让	60
9.3.1	V1: 账户模块、剧情引擎与开发方式	61
9.3.2	V2: 剧情接口文档与展示适配	61
9.3.3	V3: 需求文档与剧情设定定稿	61
9.3.4	V4: 剧情文本定稿与数据清理	62
9.3.5	回看这三周	62
9.4	高冰轩	63
9.4.1	V1: 从单个功能入手, 第一次面对团队接口	63
9.4.2	V2: 从“把内容放上去”走向“让玩家顺畅地操作”	64
9.4.3	V3: 从负责一块页面, 走向负责一条能跑通的链路	64
9.4.4	V4: 把三个小游戏从占位入口做成可交付的玩法	64
9.4.5	回看这三周	65
9.5	卢正松	65
9.5.1	V1: 认领探索与成就, 从头学协作	65
9.5.2	V2: 先对齐共识, 再动手联调	66

9.5.3	V3: 探索铺开与联调把控	67
9.5.4	V4: 对话框、网站与版本收尾	67
9.5.5	回看这三周	68
十、	评优申请	69
10.1	模块边界: 从文档约定到代码约束	69
10.2	状态一致性: 唯一写入口与事务提交	71
10.3	内容演进: 数据与引擎的分离	73
10.4	演出层: 不侵入的旁路	75
10.5	共享内核与跨模块结果交接	76
10.6	关键设计取舍	78
10.7	交付完整度	79
十一、	最终得分	80
11.1	个人评分与理由	80
11.1.1	罗晨菲	80
11.1.2	杨梦	80
11.1.3	于知让	81
11.1.4	高冰轩	81
11.1.5	卢正松	81
11.2	关于统一评分的说明	82
11.2.1	四个版本首尾相接, 没有松懈的间歇	82
11.2.2	每个人都身兼数职, 且不止于开发	82
11.2.3	模块互相咬合, 任何一环停下都会卡住整条链路	82
11.2.4	工作量有据可查	82

一、项目概述

1.1 游戏简介

1.1.1 概述

《借灯》是一款用原生 HTML5、CSS3 与 JavaScript 写成的浏览器端交互式悬疑叙事游戏。项目不依赖任何前端框架，也没有构建步骤，源码即产物。开发过程按企业级轻量化项目的规范组织：需求先落成文档、拆成 subPR 再动手，代码走功能分支与 PR 评审，五名成员在四个版本中轮换承担项目经理、产品经理、技术负责人等职责。

故事发生在虚构的皖赣交界废弃矿山古村石涧村。村里流传着一条禁忌：死在外地的人若找不到归途，会在雨夜来借一盏没有火焰的白灯笼；灯不能碰，门不能开，更不能回应门外叫自己名字的声音，否则死人会借走你的脸。玩家扮演一个在暴雨后从祠堂醒来的失忆者，身上只有一张烧毁大半、仅剩姓氏可辨的工作证和一颗蓝玻璃珠。他要在查清村庄闹鬼真相的同时，回答一个更麻烦的问题：我是谁。

项目想在三个方向上都有所交代：工程上有一套可追溯的规范流程，叙事上有一个经得起反复推敲的反转结构，主题上留下一点值得回味的感受。



图 1.1 石涧村设计图

1.1.2 核心玩法循环

游戏的基本循环是一条闭环，玩家的每一步操作最终都落在同一份进度记录上：

阅读剧情 → 探索场景 → 调查与对话 → 收集线索 → 完成小游戏 → 作出抉择

走完这一轮，剧情推进到下一个节点，循环重新开始。

阅读环节按段落推进剧情，对话带发言者名牌，独白与旁白分开渲染。探索环节里，场景中的发光热点是唯一的交互入口，点击物品或人物会触发调查、对话或线索详情，得到的物证进入背包。小游戏嵌在剧情节点上，复原手绘地图、拆解控制室网络、还原矿井路线和逃离数据机房都在此列，结果会回写到剧情状态。

1.1.3 创作背景与理念

故事来自小组的原创悬疑剧本，表层框架借用了南方山村雨夜借灯的民俗传说，氛围落在废弃矿山古村的落寞上，重心则始终放在人物的命运纠葛和选择上。

石涧村的这条禁忌是完整的：死者在外地找不到归途，会在雨夜借一盏没有火焰的白灯；活人若开门回应，脸就会被借走。它既是恐怖氛围的来源，也是叙事的机关。白灯笼、广播里的童谣、只有进没有出的湿脚印、墙上写着的已故村民名字，这些在前半段看起来无法解释的现象，故事进行到中段会告诉玩家它们全都有物理成因。真正推动悲剧的，是集团、村民、父亲和主角各自作出的选择。白灯象征的不是超自然力量，而是人借传说掩盖责任。

主人公的一生是一条闭合的因果链，每个阶段的决定都在为最后的结局埋线。

童年时矿难夺走了他的亲人，家庭随之破碎。他选择逃离故乡，毁容改名，把过去整个切开，以为这样就能重新活一次。创伤并没有因此愈合，只是被压到了意识深处。

成年后他被迫回到故地，故乡的一草一木把深埋的恐惧全部唤了回来。他没有选择直面，而是照着童年的记忆复刻出一套恐吓装置，把自己最怕的东西变成了威胁别人的工具。

为了掩盖罪行，他又分裂出白灯客这个匿名身份，抹掉作恶的证据，试图在道德上把自己和做过的事切开。这个身份既是自保的手段，也是人格崩塌的标志。

全程的沉默并没有换来救赎。他越是逃避，越是被迫面对；越是沉默，罪责越重。自我救赎的失败，构成了整条命运逻辑的闭环。



图 1.2 主角的矛盾人生设计

1.2 体验定位

1.2.1 玩家画像

《借灯》面向的是愿意花两三个小时读完一个完整故事的人。这一版预计游玩时长三小时左右，节奏由文本和调查推动，不设操作门槛：V2 之后核心操作只剩鼠标点击，键盘只用于少量快捷键。

游戏不要求玩家事先了解任何背景知识，也不依赖前作。三小时的流程里，前三分之一用来建立悬念，中间用来推翻玩家已经建立的判断，最后一段留给一个没有标准答案的决定。

1.2.2 沉浸式体验

沉浸感的建立方式，主要是在几个地方不给玩家退路。

开场失忆让玩家和主角站在完全相同的起点。你不知道自己是谁，村民对你的称呼也互相矛盾：多数人认定陈晋年十七年前就死了，只认识来谈拆迁的王工程师；一个记忆混乱的老人会突然喊出陈晋年的名字。玩家手里的信息、心里的疑问，和屏幕里的主角一模一样。

游戏里没有太多靠突然的画面和音效吓人。恐怖感来自另一件事：中段揭露所有灵异现象都是人为装置之后，玩家会认出这些装置的元素全部来自主角的童年记忆。我们希望做出这样“可怕的不是鬼，而是人”的艺术反转效果，从而引发玩家的思考与喜爱。

场景细节同样在为这件事服务。檐下的雨声、排水渠里流动的水、广播里带着电流杂音的童谣、雨后只有进没有出的湿脚印，每一样都同时属于村庄的日常和恐吓装置的一部分。

界面上则尽量不打断玩家。同一时刻只允许存在一种主状态，线索详情、背包或小游戏窗口打开时，背后的场景会被锁住，关闭后回到原来的状态。

1.2.3 感受主旨

故事表面上问的是村里有没有鬼，中段变成谁在装鬼，后段揭晓装鬼的人就是主角。真正的问题留到了最后：他为什么需要一个鬼。

只要存在一个外在的、说不清的力量在作祟，他就不用面对另一件事，换句话说，这个村子里所有的苦难，都是活人做出来的选择。

终局的处理方式呼应了这一点。玩家不是从几个结局里选一句话，而是决定保留哪几组证据、删掉哪几组。当玩家成功逃离小周之后，销毁全部、选择性公开、完整公开三条路分别通向《无名者》《白灯之后》《不再借灯》，分歧在于玩家愿意承担多少。完整公开意味着同时承认十七年前的矿难、集团的罪行、父亲的隐瞒，以及自己设计恐吓、拒绝施救和删改证据的全部事实。

我们希望玩家在通关之后，对沉默和逃避这两个词多一点警觉，这是这个项目最想留下的一点东西。

1.3 项目目标分期与实现边界

1.3.1 分期实现

项目分四个版本迭代，每一代有明确的交付范围和明确的不做清单。

V1 解决的是有没有的问题，从零到一。站点能跑，剧情能串联，核心是验证可行性。这一代以故事大纲中的 14 个探索阶段作为全局口径，V1 只完成前三个：祠堂醒来、村口开放、陈家老宅。同时交付登录、注册、游客三种进入方式，主菜单、成就、项目介绍、制作组以及五名成员各自的独立页面，共计八个以上 HTML 页面。小游戏做了复原手绘地图这一个，成就系统做到能解锁第一项，存档先只存在浏览器本地。V1 明确不做的部分，向后的剧情，第二个小游戏、完整证据链、全部身份反转和多结局等等。

V2 不新增任何剧情，只重做游戏内的交互方式。界面被拆成五种互斥状态：看剧情、逛场景、看线索详情、看背包、玩小游戏。同一时刻只能处于其中一种，弹窗打开时背后的场景锁死。探索从列表式操作改成直接点击场景里的发光热点。这一版还统一了进度来源，storyCheckpoint 成为唯一的剧情进度记录，其他字段只作为由它生成的只读投影存在，避免全局、剧情和探索各记一份互相打架的进度。

V3 把故事推到结局。老宅不再是整局的终点，而是后半段调查的入口。这一代新增 28 个运行时 Node 和四个后续剧情阶段，把三条身份线的外围调查接入正

式流程，新做三个小游戏，并把五个结局全部跑通。它的技术前提是 V2 必须先验收并合入 main，V3 分支从包含完整 V2 的最新 main 上重建，不允许在缺失 V2 的旧分支上批量开工。

V4 解决的是好不好问题，不再增加结构性内容，做的是内容改写与体验收尾。剧情对话按 46 个改写块全面修订，覆盖序幕到终局的全流程；对话框补上响应式；新增 11 处转场效果；插入过场视频；菜单页、成就页和背包页的展示细节陆续调整。这一版还处理了一个具体的渲染问题：探索热点被点击后可能走阅读视图或物品详情弹窗两条不同的路径，两条路径对多行文本和发言者名牌的处理规则并不相同，写文案时必须先判断这段话会走哪条路。

表 1.1 四个版本的目标与交付

版本	版本目标	交付效果
V1	搭起站点骨架，跑通前三阶段	8 个以上独立页面；登录、注册、游客三条入口；祠堂、村口、陈家老宅可连续游玩；复原手绘地图小游戏；最小可用成就系统；本地存档
V2	重构游戏内交互，不新增剧情	五种互斥界面状态；点击发光热点探索；storyCheckpoint 成为唯一进度来源；1280px 与 390px 双档验收
V3	从 week-one-end 推进到正式结局	28 个运行时 Node；四个后续剧情阶段；四条外围调查线；三个新增小游戏；五个稳定结局
V4	剧情改写与体验打磨	46 处剧情对话改写全部落地；小游戏二期；对话框响应式；11 处转场效果；过场视频插入；菜单、成就与背包页细节调整，视听与美术效果一致性

1.3.2 实现难点与挑战

反转节奏是最难控制的一件事。游戏的核心是三重身份的逐步揭露：先让玩家确认所谓闹鬼是人为的，再确认主角就是设计这套装置的王阙，接着确认王阙就是十七年前失踪的陈晋年，最后确认匿名举报的白灯客也是同一个人。四次揭示必须按固定顺序发生，任何一次提前泄底，后面几步就全部作废。大纲评审时反复调整过村民的台词，把原本多人直接叫出主角名字的写法删掉，只留一个记忆混乱的老人偶尔喊出陈晋年的名字，多数村民必须明确认为陈晋年已经死了。

故事逻辑的自洽花了比预想更多的时间。大纲修订清单里列出的问题大多属于这一类：搜救队第二天就找到两个孩子，和陈晋年昏迷两天后才被采药人发现，这两件事不能同时成立；妹妹事故的逐日时间线前后指代不清；陈晋年在早期稿子

里被写成冻结在十七岁，但他其实是十七岁的兄长，坠入采坑的是妹妹。父亲的动机也重写过，最初只写他远远看见白布却不敢确认，心理因果太弱，改成了他既受过胁迫、也主动毁过证，这样他后来十七年不寻找可能生还的儿子才站得住。

技术上的主要矛盾是状态一致性。剧情、探索、对话、背包、成就各自都有进度要记，如果每个模块各存一份，迟早会出现同一次操作在不同模块里结果不一致的情况。V2 的解决办法是收敛到单一进度源，并规定新代码不得读写兼容字段来决定剧情推进。协作层面同样靠约定约束：接口先给真实输入输出，双方共同跑通才算接入；页面结构、剧情文本、热点规则、场景美术、状态与存档分别归属到人，问题按归属回退；开发者不能做自己 PR 的唯一审查人。

还有一类困难来自恐怖规则要同时是玩法规则这条要求。白灯、广播、湿脚印和墙上姓名都成立，但每种装置都必须有供电范围、触发条件和可以被玩家反向利用的缺陷，否则操作者就显得无所不能，恐怖也就失去了张力。拆解控制室网络那一段的设计正是围绕这条原则展开的：玩家要逆向追踪三种异象的来源，利用延迟或供电盲区躲过下一次触发。

团队的每一位成员都在这个项目中投入了超出课业要求的心力。我们反复推敲每一句对白的分寸、每一个反转的节奏、每一条命运线索的闭环，因为我们确实相信这个故事值得被好好讲出来。我们未必做到了完美，但我们做到了诚实。如果玩家在走出这个故事之后，能对沉默和逃避多一分警觉，对面对多一分勇气，那便是我们最大的回馈。

二、游戏设计详解

2.1 主线剧情

2.1.1 开场与物品

玩家从祠堂的地上醒来，身上只有三样东西：一张烧毁大半、只剩姓氏的工作证，一颗蓝玻璃珠，一把没有标记的老钥匙。守在一旁的年轻人自称小周，告诉他你是公司外聘的逻辑分析师，来调查村里的闹鬼事件，前几天摔伤了头。

这句话只有一半是真的，而玩家此刻无从判断是哪一半。**失忆是整套剧情的启动装置：玩家与主角站在完全相同的信息起点上，村民对同一张脸给出互相矛盾的称呼，有人叫他陈晋年，多数人只认识来谈拆迁的王工程师。**玩家要回答的第一个问题因此不是村里有没有鬼，而是我是谁。

三件开场物品各自是一条身份线的入口。工作证把玩家带向项目档案室，钥匙打开陈家老宅的门，蓝玻璃珠后来在邻镇旧诊所的登记抽屉里找到同款。三条线由身上的物件自然牵引出来，玩家不会觉得自己在完成一份任务清单。

2.1.2 三条身份线

剧情采用一个村庄中枢、三条身份支线、两次强制汇合、一个矿井终局的半线性结构。村口是唯一的中枢，玩家在这里反复出入，把三条线的进度带回同一个地方。

三条线各自回答一个问题。陈晋年线回答我从哪里来，经由旧家、坠沟处、空坟和诊所档案，确认十七年前的兄妹坠沟与父亲的签字。王阙线回答我做过什么，经由村委旧楼、项目部、矿区控制室，确认恐吓系统与封井任务出自谁手。白灯客线回答我删掉了什么，经由拒签户收到的匿名信、废弃邮电所和匿名藏点，确认那份举报在哪些地方被有意留白。

三条线可以交换探索顺序，但交汇点固定。第一次汇合把所有灵异现象的物理成因摊开，第二次汇合还原苏禾之死。玩家在两次汇合之间获得的不只是线索，还有一份越来越难维持的自我解释。

2.1.3 五次揭示顺序

全篇的悬念由五次揭示推进，它们必须按固定顺序发生。任何一次提前泄底，后面几步会一起作废。



图 2.1 剧情结构总览

1. 借灯现象可以人为制造。
2. 主角就是恐吓系统的设计者王阙。
3. 王阙正是十七年前失踪的陈晋年。
4. 白灯客也是主角，而且他的举报刻意避开了自己。
5. 小周把苏禾推入井中之后，王阙明知她还活着，仍然亲口选择了不报警。

前四次是对玩家认知的推翻，最后一次把矛头转向主角的道德选择。至此玩家已经知道自己操控的这双手做过什么，剩下的问题与推理无关，只与意愿有关。

为了让这个顺序守得住，村民的台词和档案的可辨识度都做了限制：多数人必须认定陈晋年已经死了，只有一位记忆混乱的老人会偶尔喊出这个名字；王阙的正脸资料在档案里始终被剪去；白灯客从不出面，只以变声录音、匿名信和统一的灯形印记存在。

2.1.4 五结局设计

终局不给玩家一句立场选项，而是让玩家实际处置手中的证据材料。五条路径通向五个稳定结局：交出证据对应《共犯的终点》，追逐失败对应《封井之人》，销毁全部对应《无名者》，选择性公开对应《白灯之后》，完整公开对应《不再借灯》。

销毁全部通向《无名者》，村庄在官方记录里继续什么也没发生。选择性公开通向《白灯之后》，玩家保留矿难与集团违法的部分，删掉王阙设计恐吓、拒绝报警和白灯客删证的记录，以揭密者的身份获利，完成又一次责任切割。完整公开通向《不再借灯》，四组材料一起放出去，主角失去所有干净的身份，第一次以同一个名字承担全部选择。

此设计的用意在于，三个结局的差别不体现在态度，而体现在玩家愿意删掉什么。到这一步，玩家做过的事和主角做过的事是同一件事，这也是项目的核心设问

2.2 探索场景

2.2.1 场景即界面

探索部分放弃列表式菜单，改用整张场景图作为交互载体。全篇共十六余个场景，覆盖村庄中枢、三条身份线、三处玩法场景和五个结局画面。

场景按统一规范绘制与接入，画布 1280×720，16:9，满屏不裁切，图片与热点同步缩放；可调查的对象一律做成场景里的热点，不做平铺网格；单个点击目标不小于 44×44 像素，热点中心间距不小于 144 像素，保证在 390 像素宽的窄屏上依然点准；同一台面超过三个热点时，改为一级热点加放大层。

在此规则约束下，玩家在场景里看到的每一样东西，都可点、可问、可拿，场景美术因此不能只是背景板，它同时是线索的容器。

场景中的对象在被调查后会改变状态，或在视觉上留下痕迹，或标记为已调查，玩家不会重复拿到同一条线索，也不会有漏掉某个对象就必须重开一局的焦虑。

2.2.2 场景清单

表 2.1 探索场景清单

场景	所属线路	主要交互内容
祠堂	序章	检查伤口与三件随身物品，在白灯出现后寻找出口
村口	中枢	向村民打听，记录彼此冲突的说法，取得村庄手绘图
山腰旧运输道	陈晋年线	按旧送葬路线寻找坠沟处，判断两人滚向不同深度
陈家老宅	陈晋年线	用刻名钥匙开门，核对照片、校服、刻痕与送葬名单
衣冠冢	陈晋年线	确认墓中没有遗体，找到邻镇诊所的票据编号

接下页

(续表 2.1)

场景	所属线路	主要交互内容
村委旧楼	王阙线	恢复档案室电脑，查看拆迁名单、恐吓执行表与被遮去姓名的背景调查
临时项目部	王阙线	读秘密封井任务单，看到旧井写成未登记安全隐患
废弃小学	苏禾线	还原她对电线、广播与排水设施的巡查
旧矿图藏点	苏禾线	取出父辈证词、原始矿图复印件与封井材料
废弃邮电所	白灯客线	收集灯形印记的条款提醒、录音与匿名举报草稿
矿区旧控制室	王阙线	拼合系统图纸，读取调试日志与低清录像
邻镇旧诊所	陈晋年线	用票据编号查到账簿、伤情图与无名病人转诊记录
父亲旧屋	父亲线	进入暗层，找到爆破日志、威胁材料与未寄出的证词
匿名藏点	白灯客线	开启藏点，比对原始文件、举报版本与删除记录
广播室	玩法	拆解控制室网络小游戏，逆向追踪相关恐怖故事的来源
旧矿井	玩法	还原矿井路线小游戏，绕过封堵进入竖井
数据机房	玩法 / 终局	对峙追逐小游戏，取得记录并决定证据处置方式
村口白灯	结局	五个结局

2.2.3 人物对话

不同角色说不同的话是核心塑造规则。小周冷静克制，从不用感叹句，也很少直接否定主角，在关心主角的同时显露出一些异常；村民保留口语和方言味，同一件事各有各的说法；主角的内心独白短，且多数以疑问收尾。

信息有分寸。村民群体被拆成有差别的记忆持有者：拒签户、已经搬走的人、当年收过补偿的人、仍然相信借灯禁忌的老人，对十七年前的事故各有各的版本。玩家在村口收集到的说法互相冲突，冲突本身就被设计成线索。

2.2.4 背包与线索

调查得到的物证进入背包，线索独立成页，两者分开管理。玩家拿到的是物证，理解的是线索，证据链由线索之间互相印证构成。

三条身份线各自提供的证据能证明的事情是有限的，设计时刻意保留了这一点。陈晋年线的照片、票据与空坟记录能证明兄妹坠沟和陈晋年生还，证明不了集团的违法；王阙线的背景调查与调试录像能证明恐吓系统出自谁手，证明不了他事

前知道旧事故的真相；白灯客线的匿名信与删改记录能证明泄密时回避了自身责任，不能等同于完整自首。任何一条线单独拿出来都推翻不了全部假设，玩家必须把三条线的证据拼在一起。

2.3 核心交互

2.3.1 剧情对话

旁白、系统提示和人物对白统一走阅读组件。组件负责逐段显示、控制继续与关闭；剧情模块决定说什么、能做什么；对话模块负责具体的角色对白。三层分开之后，改一句台词不需要动流程，加一个流程节点也不需要重写文案。

对话框在窄屏下换行重排，发言者名牌与正文分行渲染，多行文本不会顶掉名牌。人物立绘按情绪切换，配合字号与留白的变化，让长时间阅读不至于疲劳。

V2 之后游戏不设影响主线的分支选项。玩家只在剧情明确给出操作时才能提交选择，普通的继续阅读不会改变任何状态。这条限制是刻意的：故事要的是玩家沿着一条确定的因果链走到底，而不是在岔路口反复权衡。

2.3.2 四种小游戏

小游戏嵌在剧情节点上，玩完把结果回写给剧情模块，由剧情模块决定后续走向。四个小游戏分别承担不同的叙事功能。

复原手绘地图放在村口，玩家从三名村民处取得地图碎片后拼成完整村庄图，拼好即解锁通往陈家老宅的路径。它教玩家读懂这个村庄的空间关系。

拆解控制室网络安排在第一次汇合。玩家要逆向追踪童谣、白灯和湿脚印的来源，关闭其中一组装置。每种装置都有供电范围、触发条件和可以被反向利用的缺陷，玩家利用延迟或供电盲区躲过下一次触发。恐怖规则在这里正式成为玩法规则，也是全篇唯一一处让玩家主动拆解恐惧的地方。

还原矿井路线发生在旧矿井。玩家依照原始矿图从排水洞绕开封堵，检查两道挡墙、废石填层和速凝浆的痕迹。这一段的目的是让玩家亲手确认一件事：当晚封掉的只是唯一公开入口，竖井从来没有被灌满，井下的人本可以被救出来。

逃离数据机房放在数据机房，详见本章第四节。

四个小游戏都不以难度取胜。它们是让玩家在关键节点上动手，而不是让玩家卡关。

2.3.3 恐怖表现与设计

恐怖表现集中在三类手法上，都服务于同一件事——让玩家相信有什么东西在这个村子里。

音效是其中最重要的一层。檐下的雨声和环境底噪持续存在，广播里的童谣带着电流杂音定时出现，远处传来爆破的回声。这些声音同时属于村庄的日常和恐吓装置的一部分，玩家在中段之前无法分辨，中段之后回头看，会发现每一处声音都解释得通。

画面层面用黑屏、花屏和抖动处理关键转场，全篇共十一余处。它们的共同规则是只出现在状态切换的瞬间，用来制造一次短暂的失控感，不侵入阅读区域，不影响正在显示的文本。祠堂断电那一段是其中最典型的一处，灯灭与转场叠在一起。

过场视频用在几个无法用静态场景交代的位置，配合白灯 CG 与画面闪黑衔接前后场景。人物的情绪转折、时间的跳跃、回忆片段的插入都交给视频处理，避免用大段旁白去解释画面。

所有恐怖手法的边界是明确的：不中断操作，不遮挡文本，不靠突然的巨响吓人。真正的压力来自玩家逐渐意识到自己拆解的每一个装置都眼熟。

2.3.4 存档与进度恢复

游戏状态保存在浏览器本地，包含剧情检查点、已确认事实、调查进度、对话进度、物品、线索、地点、地图拼图结果、选择和成就。

剧情检查点是唯一可信的进度来源。页面、探索、对话和小游戏都不能自行修改当前节点，一次操作只有在状态提交和存档成功之后，才会在界面上显示为完成。这样做的收益在协作阶段体现得最明显：五个人分头开发，谁改动一个模块都不会让另一处的进度对不上。

注册用户和游客使用各自独立的本地存储空间，互不覆盖。系统分别处理没有存档、存档损坏、版本不兼容和浏览器无法写入四种情况。恢复失败时保留原始数据并如实告知玩家，不会悄悄当成一局新游戏重新开始。

2.4 附加功能

2.4.1 逃离数据机房与生命值系统

设计的终局小游戏是数据机房主题追逐关卡，玩家需躲避小周追捕，收集三份关键证据并撤离出口。关卡引入生命值系统，强化追逐生存压力；受击后提供 1 秒无敌时间，防止连续碰撞瞬间暴毙，平衡游玩容错。对斜向移动做速度归一化处理，走位操控更加稳定流畅。证据具备防重复计数逻辑，关卡成败结果可回传主线，联动整体剧情。俯视视角角色立绘搭配密闭机房场景，便于观察敌人动向，进一步烘托出悬疑紧迫的逃亡氛围。

2.4.2 成就系统

成就页共收录 14 条成就，按剧情推进的层次分成三组：调查札记 3 条，灯下真相 6 条，终局余烬 5 条。页面标题是雨夜留痕，顶部给出解锁进度与一条进度条，下面是三组成就卡片。

每条成就的解锁条件不是另写一套判定，而是直接声明它依赖哪些已经登记的剧情事实。复原手绘地图完成对应残图归一，确认异象系人为制造对应灯下无人，三重身份合并对应三名照一身，逃离数据机房生还对应雨幕留痕。成就因此不需要维护第二份进度，剧情走到哪里，成就就跟到哪里。

表 2.2 成就目录

分组	成就	解锁条件
调查札记	残图归一	完成村口复原手绘地图
	尘封仍有回声	走完陈家老宅一段并确认前后说法矛盾
	雨线交汇	四条外围调查线全部走完
灯下真相	灯下无人	确认白灯与童谣系人为布置
	三名照一身	陈晋年、王阙、白灯客三条线合并
	封井之外	绕过灌浆封堵进入旧矿道
	迟来的证词	还原苏禾之死的完整链条
	灯尽见真章	四组证据全部集齐
	雨幕留痕	在逃离数据机房中生还
终局余烬	灯火照旧	结局一《共犯的终点》
	井口又高一寸	结局二《封井之人》
	姓名落尽	结局三《无名者》
	借来的黎明	结局四《白灯之后》
	此后无须借灯	结局五《不再借灯》

五条结局成就被标记为隐藏条目。未解锁时，成就页只显示未明之章配合罗马数字序号，描述文案完全不渲染。这条规则是为了防住一个很容易踩的坑：成就页是最容易被玩家提前翻看的地方，如果把五个结局的名字直接列出来，整条主线的走向就白送了。成就页因此对未解锁的结局保持沉默。

卡片左侧用一个字区分状态，已解锁显示灯，未解锁显示影。规则模块判定出某条成就可以解锁，不等于它已经解锁——事件要由状态模块按游戏局、存储范围和幂等键提交成功，卡片才会点亮。同一份成就模块只允许一个视图负责新解锁

提示，避免同一件事被提示两次。

数据异常被限制在单条记录内。某条成就缺少对应剧情事实，或解锁时间格式无效，只有该条显示记录异常，其余卡片和整个页面的生命周期不受影响。读档失败也按原因分别处理：存档损坏、版本不兼容、登录身份与存档不匹配、浏览器存储不可用各有各的提示；没有存档不算失败，页面会显示当前还没有游戏存档

2.4.3 项目组信息

制作组与项目介绍分成两个页面，五名成员另各有独立页面，页面之间互相链接。制作组页面的内容分三块：团队概览给出组名白灯客、项目名与版本号，以及一句团队自述；成员档案列出五人的头像、角色与一句话简介；项目分工按版本记录每个人做过什么。

这些资料全部集中在一个数据文件里，HTML 只管渲染，不在页面里硬编码成员信息。这条约定的好处在改稿时最明显：调整某位成员的分工或换一张头像，只需要改数据文件的一个字段，五个页面同时生效，不会出现某个页面还是旧版本的尴尬。

成员页面同样按数据驱动的方式渲染。每页有档案编号、姓名缩写标记、头像、角色、一句话简介、个人自述与个人引语。字段为空时不显示，而不是显示待补充的占位文案——这条原则写在了数据文件的文件头，成员没有提供的资料，页面就空着，不替他编。头像加载失败时该图片元素直接移除，不会在页面上留下一个破图。

表 2.3 项目分工（按版本）

成员	各版本分工
罗晨菲	V1 全局、主菜单、页面导航与存档；V2 全局状态、存档、错误处理与正式集成，需求文档撰写；V4 全局状态协调，素材、视频、音效接入
杨梦	V1 入口与核心视觉；V2 场景美术、素材与视觉验收；V3 场景美术、视觉、音效与剧情回归；V4 转场素材、剧情视频、音频素材及接入
高冰轩	V1 小游戏、项目介绍与制作组页面；V2 游戏页面、统一阅读组件与响应式；V3 全局协调、状态存档；V4 三个小游戏的设计与完善
卢正松	V1 场景探索、NPC 对话、背包与成就；V2 热点、对话、线索、背包与成就；V3 点击式探索与整体功能集成；V4 对话框设计、物品详情卡片与菜单页调整
于知让	V1 账户与剧情模块，需求文档撰写，剧情初稿确认；V2 剧情 Node、剧情展示与剧情回归；V3 需求文档撰写，V4 剧情提示与 NPC 对话完善，文字悬疑深入

分工按版本而不是按模块记录，是因为这个项目里每个人的职责一直在换。五

个人在四个版本中轮换承担项目经理、产品经理、技术负责人等职责，同一个人在不同版本里做的可能完全不是一件事。按版本逐条写下来，谁在哪一代交付了什么就一眼可见，期末回看也不会把两个版本的工作混在一起。

2.4.4 音效与美术

音效与美术围绕同一套听觉与视觉规范展开。场景美术的配色限定在冷夜、连阴雨、雾和白灯暖光四个方向上，白灯是全篇唯一使用冷青高光的物体，其他光源一律压低。这套限制让玩家在昏暗的场景里始终能一眼找到该点的东西，也让白灯在视觉上始终是例外。

音效负责环境层，美术负责信息层，两者共同的底线是不干扰阅读。具体的视听规范、色彩体系与界面设计见第三章。

这一章写的都是我们最终要做成的东西。四个版本迭代下来，我们把故事结构、场景规则、交互层级和视听手法全部拆开重排过不下五次，为的是让玩家在三个小时之后，回头能找到每一个细节的出处。我们未必把每一项都做到了最好，但每一项为什么这么做，每个人都很清楚。

三、美学与用户体验

3.1 视觉设计规范

3.1.1 整体风格

《借灯》的视觉只讲一个对比：冷夜里的那盏灯，基于强烈的冷暖对比进行设计。

故事里的白灯笼没有火焰，却在雨夜里泛着白光。它代表掩饰，在画面中被做成全篇唯一的暖光源。其余元素，雨、雾、废弃的矿区、没有窗的屋子，全部压在冷调里。玩家进入任何一个场景，视线都会先落到亮的那一点上。

这个对比同时解决了一个实际问题。全篇场景出自不同批次，祠堂在室内，矿井在地下，数据机房是现代的，如果各画各的，玩家每换一个地方都要重新适应。规范因此固定四样不变的东西：色板、雨、灯、字体。场景内容可以不同，这四样必须一致，玩家切换场景时感受到的是同一个雨夜里的不同地点。

3.1.2 配色

全站颜色收拢到一个变量文件里，其余样式表只引用变量名，不单独书写色值。这样做的目的是让配色一致性自动维持：修改一处底色，所有页面同步变化，不会出现主菜单与老宅场景相差三个色阶的情况。

用色规则可以概括成一句：底色永远冷，强调永远暖。

朱砂红是使用限制最严的颜色，只允许出现在三处：关键真相线索、错误提示、门外那个叫名字的声音。一旦大面积使用，它就不再表示紧急，玩家对它的警觉也会随之消失。

两项可读性指标定为硬性要求：正文与背景的对比度不低于 4.5:1，最小的楷体字不小于 12px。探索区叠加在场景图上的标签必须垫一层深色半透明底，不能把浅色字直接压在亮背景上。场景图中存在高亮区域，浅色标签叠加其上会失去可读性。

3.1.3 字体

宋体笔画清晰，适合长段落阅读，用于剧情正文和场景名。楷体接近手写，放在说明性小字上，与正文形成区分，避免提示文字打断阅读。按钮和顶栏需要快速识别，使用无衬线字体。

表 3.1 全站色板

分组	色块与 Token	用途
冷底	 --bg (#07090e)	页面最深底色，场景最暗处
	 --bg-elevated (#0d1118)	面板与卡片底
	 --panel	文本框与侧栏的半透明底
纸与暖金	 --lamp (#f5f0e3)	白灯的光，热点光晕与高光
	 --paper (#ece7dc)	正文纸白
	 --paper-dim (#cfc8b8)	次级文字、旧纸
	 --accent (#d8c6a4)	强调暖金，木纹、旧物、描边
	 --accent-deep (#b89b6e)	深金，按钮悬停态
冷点缀	 --mist (#8fb8ce)	雾蓝，冷光与次级点缀
	 --rain (#5d7f9e)	雨蓝，雨丝与冷调
警示	 --cinnabar (#a6534a)	朱砂红，关键真相与门外的声音
	 --danger (#c17a74)	错误与警示

表 3.2 字体与字号阶梯

层级	字体	字号 / 行高	用途
标题	宋体	24px / 字距 0.18em	游戏名、章节名、场景名
正文	宋体	17–21px / 1.9	剧情正文、人物对白
旁白与小字	楷体	12–15px / 1.6–1.8	说明性小字、旁白细节
界面标签	无衬线	14px	按钮、顶栏、系统提示

封面单独使用一套毛笔手写字体，全篇只在封面出现一次，重复使用会削弱它的辨识作用。

正文的 1.9 倍行高和标题 0.18em 的字距是依据中文排版特点设定的。中文没有词间空格，字号越大、字距越密，长句越容易读串行，行高和字距是替代空格的主要分隔手段。

3.1.4 人物立绘

立绘只有两个站位，左和右，分别落在画面两侧 5% 的位置。哪个角色站哪边写在数据里，渲染层只读站位数值，不判断来的是谁。这样加一个人物不需要改渲

染代码，改一个字段就够了。

立绘的作用是表示场景中有人在场，同一时刻只出现一个，站位的变化即表示说话人更换。

苏禾全篇不出正脸立绘，只以笔迹、照片和登记记录出现。这是因为，她的死要等到第二次汇合才揭晓，任何一张正脸立绘都会让玩家在她真正出场之前就先认出她。美术不必把每个角色都画出来，隐藏不该出现的角色同样是设计的一部分。

3.1.5 美术原则

场景图同时承担背景和线索载体的功能，美术有四条硬性要求。

1. **光源统一。**每个场景只设一处主白灯光源，位置随场景走：祠堂在门和梁上，村口在电杆和屋檐，老宅在门后。光晕半径约为场景短边的一半到七成。任何情况下不得遮住发光热点、线索和对白区。
2. **冷暖不混。**关键对象和热点不许埋在暖光晕里。玩家点哪一处、就看哪一处
3. **画布与安全区统一。**场景按 1280×720 绘制，16:9，满屏不裁切。底部 600 到 720 一条留给阅读框和按钮，关键信息不往里放。
4. **背景不许剧透。**每张图交付前要过一遍：这张图有没有提前泄露某个人的结局、某一层身份的关系。有就改。

这四条写进了场景交付模板，每张图由美术、剧情和前端三方逐项确认之后才能合入。氛围统一因此有了可以逐条核对的验收标准。

3.1.6 动效规范

日常动效控制到几乎不可察觉，只有三类。

雨丝垂直缓慢下移，透明度很低，不干扰阅读。白灯的光晕在 0.85 到 1 之间以约 2.5 秒的周期缓慢起伏，该效果只用在灯和发光热点上。场景切换统一使用 200 到 400 毫秒的淡入淡出，全篇不做变化。

限制强度的原因是：玩家要在这个界面上读三个小时的字，任何一处抢注意力的动效都会被反复看到几百遍。动效的作用是维持画面的活性，不是吸引玩家注意动效本身。

规范中另有一条硬性要求：动效不触发眩晕，不做大面积快速位移，并且尊重系统的减少动态效果设置。高强度的动态效果全部留给关键转场，见本章第四节。

3.2 文字风格

3.2.1 整体调性

全篇有一条硬性要求：游戏里出现的所有文字都要出自故事世界内部，不出现项目组直接对玩家说话的情况。

这一条在执行时比想象中容易破。V4 定稿时删改幅度最大的一批文案，问题几乎都出在这里。原文里有一句“恭喜你，成功解锁【陈家老宅】相关剧情”，句式上是系统提示，效果上把玩家从故事里拽了出来，让他在沉浸最深的时候意识到自己在操作一款游戏。同一批改写里还有一处“归档结论”，用文件摘要的语气替玩家把结论下完了。这两处最后都换掉了：前一句整句删除，后一句改由主角的独白提出，而且停在问句上。

判断标准可以总结成一句：结论由玩家得出，文本不替玩家下结论。

3.2.2 用语

写作时要求描述现场能直接看到、听到的东西。

改稿前有一段描述证据吻合的文案，用的是重合、证明、表明这一类词。这是文件摘要的口气，读起来像在审一份报告。改写之后变成了：纸已经很旧了，可撕口还是白的，是最近才撕开的。两句话说的是同一个判断，前一种写法要玩家自己翻译一遍，后一种写法直接把画面放到他眼前。

抽象概括会让玩家觉得自己在读材料，具体的形貌、声音、温度，才会让他觉得自己站在那个地方。

角色语言的个体差异在第二章已经展开过，这里只补充它对叙述层的要求：同一件事，村民用口语，小周用短句，主角用疑问句。叙述层不跟随任何一个人的语气，它始终是全篇最冷的那一层。

3.2.3 氛围营造

氛围的营造依靠信息的分寸，不依靠形容词。

全篇没有一句直接告诉玩家这里很吓人。与其写雨夜的村子很阴森，不如写纸上那道撕口还是白的。先给出一个具体的东西，再把它留给玩家自己去理解。

名牌制度也服务于这一点。旁白不出名牌，正文自己陈述，玩家读到的是一段没有来源的叙述；独白出名牌，那句话就被标记为主角自己的声音。两者在视觉上区分得很清楚，玩家始终知道一句话是谁说的，或者知道它没有来源。

情节里最沉重的细节，我们选择不做解释。父亲签下停止搜救那一段，处理方式是让玩家自己停下来，文本不跟任何评价。这种留白是刻意的，目的是把最后一步留给玩家自己完成。

3.3 界面设计

3.3.1 站点地图

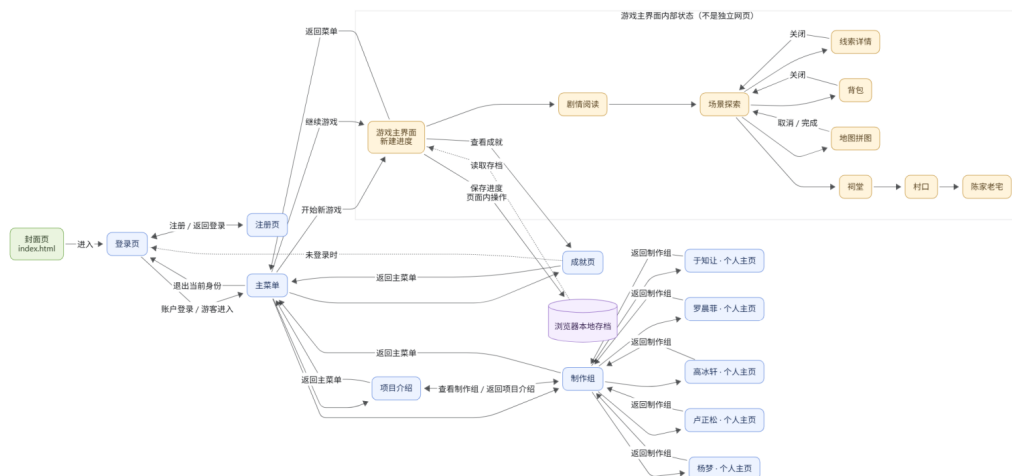


图 3.1 站点地图结构

3.3.2 线框图

【待补充】

3.3.3 设计图



图 3.2 祠堂灯亮设计



图 3.3 四条探索分支背景设计

3.4 恐怖美学设计

3.4.1 整体特点

全篇的恐怖不使用惊吓手法。

惊吓的效果只持续几秒。我们采用的方式是持续的心理压抑：玩家玩到中段时意识到，自己刚才拆开的每一个装置，材质、颜色、声音，都来自主角小时候见过的东西。那一刻不产生惊吓反应，产生的是心理上的沉重感。

因此全篇没有突然出现的巨响，主要手法都带有一层铺垫：灯先闪两下才暗下去，声音先响画面再跟上，门外先叫一声名字，隔一拍玩家才听清那是谁。恐惧感来自预期落空的那半秒，与音量无关。

这套做法的核心是反差。玩家在前半段接受的是一套民俗禁忌的解释，中段被告知所有异象都是人为布置，此时本该放松。紧接着他会认出，布置所用的元素全部取材于主角的童年创伤，他把自己最怕的东西做成了恐吓他人的工具。恐惧的来源由外部转向人本身，并且落在玩家自己操控的这双手上。

3.4.2 音效

音效承担环境层，同时负责制造真假难辨的效果。

檐下的雨、排水渠里的水、广播里带电流杂音的童谣、远处传来的爆破回声，这几层声音持续存在。它们同时属于村庄的日常环境与恐吓装置的一部分。玩家在中段之前分不出哪一层是人为布置的，中段之后回头看，每一声都解释得通。让人不安的是玩家发现自己听了两个小时，却始终没有听出其中的区别。

背景音乐只有一条循环轨，音量控制在四成左右，切换页面时不重新起播。玩

家在菜单、成就页和游戏之间来回切换，音乐不应中断。

浏览器的自动播放限制决定了第一次出声必须等待玩家的第一个操作。音频在玩家第一次点击或按键之后起播，在此之前保持静音，界面上另有一个常驻的静音开关。这项限制无法绕过，可以做的是不让它成为意外：玩家按下第一次鼠标时音乐正好响起，让它看起来是设计好的。

音频加载失败不能连带停止游戏。播放器返回失败码后，游戏降级为无声继续，控制台保留一条记录。玩家可以听不到背景音，但不能卡在黑屏前。

3.4.3 视频

过场视频用于静态画面无法交代的位置：人物的情绪转折、时间的跳跃、以及回忆片段的插入。这些位置用旁白叙述会拖慢节奏，因此交由视频承担。

全篇七段，序幕两段，五个结局各一段。

表 3.3 过场视频清单

触发位置	素材	内容
序幕 · 白灯初现	light_up	门外忽然亮起一盏灯
序幕 · 祠堂断电	cut-off	祠堂内忽然断电
结局一《共犯的终点》	the_ending	灯火照旧
结局二《封井之人》	close_well	井口又高一寸
结局三《无名者》	unnamed	姓名落尽
结局四《白灯之后》	after_whitelamp	借来的黎明
结局五《不再借灯》	no_more_lamp	此后无须借灯

触发条件绑定在台词内容上，不绑定在剧情节点的编号上，读到对应句子才播放。这样处理在协作阶段省去很多返工：节点编号会随剧情改写而变动，如果视频绑定编号，每改一次文案就要重新核对七个编号，漏掉一个就会导致一段视频永远不播放。绑定文本之后，修改剧情不会漏掉视频，增加视频也不用改动剧情数据。每条视频只播一次。

3.4.4 转场

转场是全篇唯一使用高强度效果的位置。日常动效控制到几乎不可察觉，效果强度集中在下面这十一处。

三类基本手法，加一种抖动接闪黑的组合手法，共同点是都带有铺垫，没有一处是突然出现的。

表 3.4 转场手法

手法	时长	过程
轻闪黑	0.9 s	快速压黑，短暂停顿后淡出
窒息式闪黑	3.3 s	灯闪两下 → 黑幕以 550 ms 漫上 → 黑暗中边缘红晕呼吸 1 s → 以 1.5 s 缓慢退出
暴力抖动	0.65 s	位移从 18 px 逐级递减到 0，叠加旋转与轻微模糊
无信号花屏	2.1 s	随机噪点从画面某一点扩散到全屏，短暂停顿后收回到同一点并淡出
抖动接黑	3.1 s	先抖动，120 ms 后黑幕漫上，其余同窒息式

窒息式闪黑中的两次灯光闪烁有明确作用。先铺垫再压黑，玩家在黑暗真正到来之前已有半秒的预感，恐惧感来自这半秒的预期落空，与黑暗本身无关。恢复阶段用 1.5 秒，比压黑慢了三倍，目的是让玩家在离开黑暗时多停留一会儿。

花屏的扩散和收束都从一个随机点开始，两次使用同一个点。收束速度慢于扩散，形成缩回原处的效果。

十一处触发同样全部绑定在台词上。一个监听器观察剧情文本的渲染结果，读到约定的句子就播放对应效果，剧情数据里不埋任何标记。这样处理的好处与视频一致：修改剧情不会漏掉转场，增加转场不需要改动剧情。

所有手法都绘制在场景层内部，与阅读区是两个独立的层。黑屏期间文本仍然存在，只是被遮住，恢复之后玩家可以接着读。全篇没有一处转场会顶掉正在显示的文字，也没有一处会接管鼠标。恐怖效果可以打断节奏，不中断操作。

四、技术方案与质量保证

4.1 开发与运行平台

本项目是纯前端静态站点，全部代码在浏览器端运行，不依赖服务器进程与数据库。平台选择与三周的开发周期有关：成员使用的操作系统并不一致，项目因此不能依赖任何平台特有的工具链；没有服务端与构建步骤，任何人克隆仓库之后可以直接打开页面验证效果。

表 4.1 开发与运行平台

项目	方案
开发平台	常规代码编辑器、Git、GitHub；成员可使用 Windows、macOS 或 Linux
前端技术	原生 HTML5、CSS3、JavaScript，不引入没有实际需要的框架和打包器
运行方式	静态站点，通过本地 HTTP 服务或部署后的站点地址访问
运行平台	最新版 Chrome、Edge；桌面端与移动端浏览器视口
数据保存	浏览器 localStorage，注册用户与游客使用不同存储键
版本管理	GitHub Flow，main 为唯一长期分支，功能在短期 Feature 分支开发

4.2 技术选型

4.2.1 原生技术栈

本项目不使用框架和打包器。组内成员的编程基础不一致，原生 HTML、CSS 和 JavaScript 可以让所有人立即投入开发，省去框架与配套工具链的学习成本。调试链路也是最短的：改完代码刷新浏览器就能看到结果，没有编译等待。轮值交接时，接手的人不需要先理解框架的生命周期或响应式机制。

代价是没有现成的组件复用与路由方案，这部分需要自行实现。本项目页面数量有限，全局状态集中在存档与进度上，自建的成本低于引入框架的成本。

4.2.2 浏览器本地存储

玩家进度保存在浏览器的 localStorage 中，不经过服务端。

存档键的格式为 `white-lamp:save:{身份}:v2`。身份段有两种取值，游客为 `guest`，注册用户为 `account`：加账户 ID。两种身份的数据物理隔离，游客存档不

会覆盖注册用户的存档。键名中带存档格式版本号，格式升级时不覆盖旧数据：读到旧版本的存档会提示玩家开始新游戏，旧档保留在原处。

账户信息另存于两个键，前缀同为 `white-lamp:auth:`，其后分别是账户表 `accounts:v1` 与当前会话 `session:v1`。

`localStorage` 的容量上限为每个站点 5 至 10 MB。本项目存档只记录进度、已读文本与成就状态，体积远低于这一上限。

4.3 技术架构

代码按表现层、逻辑层、数据层三层组织，三层之外有一个全局状态对象作为数据中枢。



图 4.1 技术架构图

表现层是 HTML 与 CSS。页面使用语义化标签，样式分为全局、页面、游戏三类；全站色值与间距收在 `tokens.css` 一份变量表里，其余样式表只引用变量名，不写死具体色值。

逻辑层是原生 JavaScript，按功能域拆成七个模块：核心、主线、探索、小游戏、账户、成就与介绍页。模块之间的调用关系是单向的，主线、探索与小游戏各自读写全局状态，彼此不直接引用。改动其中任一模块，不需要通知另外两个。

数据层是浏览器本地存储，读写统一经过 `core/storage.js`，带版本校验与失败兜底。剧情内容不放在这一层，它是随代码一起发布的静态数据，不写入玩家设备。

4.4 代码组织

目录结构的设计目标是让任何人打开仓库都能迅速定位到目标文件。四条规则：一个功能放在一个明确位置；页面数量可数；资源按类型分层，并在二级目录下继续按功能细分；数据与逻辑分离，全部剧情内容放在 `game-line/data/` 下，引擎只负责渲染和推进，不硬编码具体对白。新增剧情阶段时只增加数据文件，引擎核心代码不动。

Listing 4.1 项目目录结构

```

1 jiedeng/
2 +-- index.html           cover page
3 +-- pages/               12 game pages
4 |   +-- game.html       main story
5 |   +-- menu.html       main menu
6 |   +-- authorize/      login / register
7 |   +-- achievements/   achievements
8 |   \-- about/          project, team, 5 member pages
9 +-- assets/
10 |   +-- css/
11 |   |   +-- tokens.css  design tokens (single source)
12 |   |   \-- ...        per-page stylesheets
13 |   +-- js/
14 |   |   +-- core/       global state, save, nav, transitions, audio
15 |   |   +-- game-line/  story engine; data/ holds all story data
16 |   |   +-- exploration/ hotspots, items, branching dialogue
17 |   |   +-- minigames/  4 minigames + shared 2D engine
18 |   |   +-- authorize/  account, session, permission
19 |   |   +-- achievements/ definitions and checks
20 |   |   \-- about/
21 |   +-- images/         scenes, characters, items, UI
22 |   +-- audio/          background music
23 |   +-- videos/         7 cutscene videos
24 |   \-- fonts/
25 +-- test/               unit tests and browser regression
26 \-- docs/               requirements, interfaces, story, this report

```

各模块的规模如表 4.2。

样式表另有 12 个文件、3,802 行，其中 `tokens.css` 是全站唯一的变量来源。图片资源 86 个，按用途分为场景图、人物立绘、物品与界面四类；过场视频 7 段，

表 4.2 代码模块划分与规模

模块	文件	行数	职责
core	13	4167	全局状态、存档、页面控制、导航、过场与音频
game-line	13	4126	剧情引擎与全部剧情数据
exploration	22	2545	场景热点、物品与背包、分支对话
minigames	23	2428	四个小游戏与共用 2D 引擎
authorize	8	1142	注册、登录、会话与权限
about	4	690	项目介绍与制作组页面
achievements	9	579	成就定义、判定与展示
合计	92	15677	

与第三章的清单一一对应。

4.5 质量保证

4.5.1 异常与降级

原则是所有可能失败的外部调用都返回结构化错误码，由调用方转成玩家可读的中文提示，不让异常中断流程。

- **本地存储不可用。**浏览器的隐私模式或配额写满都会让读写抛错，所有读写都包在 try/catch 里，失败统一返回 STORAGE_UNAVAILABLE，界面提示“浏览器存储暂时不可用”，游戏本身不崩溃。
- **音频自动播放被拦截。**audio.play() 的失败被捕获为 AUDIO_PLAY_BLOCKED，音乐改为等待玩家的第一次点击或按键后起播，界面上另有常驻的静音开关。
- **过场视频无法播放。**播放器的结束、出错、中断三个事件绑定同一个收尾函数，play() 的失败也走这条路径。任何失败都让剧情直接继续，玩家可能看不到某一段过场，但不会停在黑屏前。
- **存档版本不匹配。**读到版本号低于当前格式的存档时返回 SAVE_VERSION_UNSUPPORTED，提示玩家开始新游戏，旧数据保留在原处不删除。

有一处刻意不做降级：小游戏的渲染循环在检测不到 requestAnimationFrame 时直接抛错停止。项目假定运行环境是现代浏览器，这里做静默兜底只会把问题推迟到更难定位的地方。

4.5.2 测试

测试代码在 `test/` 目录下，41 个测试文件、7295 行，分两层。

逻辑层用 Node 内置的 `node:test` 运行单元测试，覆盖存档读写与版本兼容、账户与权限、主线与探索之间的接口契约、小游戏规则与成就判定。

页面层用 Playwright 驱动浏览器回归脚本，脚本自带本地静态服务器，真实打开游戏页面走一遍主流程，检查存档写入与页面状态。这一层覆盖单元测试看不到的部分：脚本加载顺序、模块之间的实际联动，以及存档在真实浏览器中的落盘结果。

另有一份主线引擎的冻结副本放在 `test/exploration/vendor/` 下，探索模块针这份副本做契约测试。主线改动接口时，契约测试会在合并前失败，避免上游的改动无声地破坏下游。

详细的测试结果见第六章。

4.5.3 协作流程与 subPR 分工

版本管理使用 GitHub Flow。`main` 是唯一长期分支，始终保持可运行、可演示的状态；功能在短期 Feature 分支上开发，一个分支对应一个 subPR，只解决一个需求，合并即交付。

每个 subPR 在合入前经过技术审查与需求验收两道检查。技术审查关注实现是否清晰、有没有引入不必要的依赖；需求验收对照该 subPR 对应的需求条目逐项确认。审查在 GitHub 上进行，讨论记录随 PR 保留，后续出现问题时可以回溯当时的判断依据。

详细的成员分工与进度安排见第五章。

五、项目管理与进度

5.1 项目提出与开发方式

9 月 2 日，五名成员各自提出选题方向，逐一陈述并互相质询，当天确定最终方案，并完成仓库初始化。项目定名为《借灯》，制作组名白灯客。

9 月 3 日确定开发方式：需求驱动的 subPR 闭环。整条链路固定为五步，提出需求、编写需求文档、需求评审确定边界与实施方式、开发、测试与集成。开发方式确定之后，四个版本的迭代都按这条链路推进，不再中途改动。

5.2 协作流程

版本管理采用 GitHub Flow。main 是唯一长期分支，始终保持可运行、可演示的状态；功能在短期 Feature 分支上开发，一个分支对应一个 subPR，只解决一个需求，合并即交付。

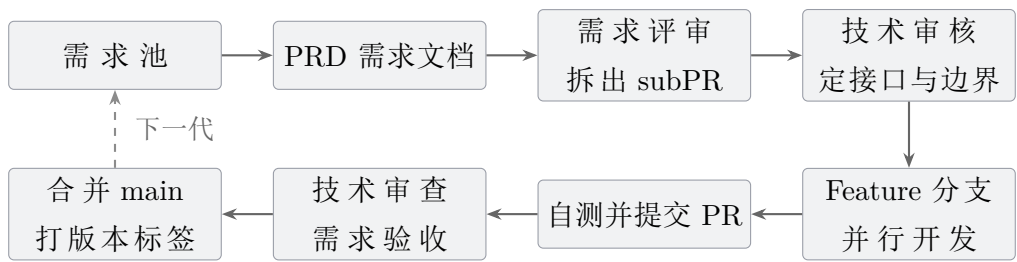


图 5.1 subPR 闭环流程

流程的每一步都有明确的输出物：需求池的条目进入 PRD 成为可评审的需求，需求评审把一个需求拆成若干 subPR 并确定边界，技术审核确定模块接口与分支策略，开发在各自的 Feature 分支上并行进行，合并前必须同时通过技术级测试与需求级验收。合并之后回到需求池，作为下一代需求的输入。

5.2.1 subPR 的运作规则

1. 每个 subPR 从最新 main 创建短期分支，命名为 feature/ 加版本、需求 ID 与简述。
2. 一个分支只服务一个需求，提交记录说明实际变化。
3. 提交评审前先拉取最新 main，解决冲突并重新完成自测。

- 4. PR 描述关联需求 ID、验收条件、测试证据、界面截图与已知问题。
- 5. 合并前同时满足技术级测试与需求级验收。
- 6. 合并后删除已完成的 Feature 分支，全部 subPR 完成后为版本打标签。

不建立长期 dev 分支；不允许在个人分支上积压到最后一天再一次性合并。

5.2.2 五日迭代节奏

一个迭代周期为五个工作日，每天的产出固定，不随人员轮换改变。

表 5.1 迭代日程

迭代日	活动	时长	核心产出
第 1 天	需求评审日	全天	确定本周 PRD，含需求清单、subPR 边界与验收条件；完成故事细节确认与素材需求梳理
第 2 天	技术审核日	全天	技术负责人确认接口定义、模块边界与分支策略；各模块负责人完成设计方案并互审
第 3-4 天	开发日	两天	各 subPR 并行开发、自测；每日 09:00 站会同步进度；每日合并前完成 PR 审查与功能验收
第 5 天	讨论复盘日	全天	汇总本周成果，验收测试，整理遗留问题与风险；输出下一代 PRD 输入材料，完成角色轮换交接

需求评审日对需求池中的待开发条目做三项决策：确认优先级（P0 阻塞项、P1 核心功能、P2 优化项）、识别依赖关系（哪些需求阻塞其他需求、哪些可以并行）、把需求拆成 subPR 并逐一分配到负责人。开发日按 P0 优先、P1 随后、P2 弹性安排的顺序推进。

开发日的站会上，每人依次回答三个问题：昨天完成了什么，今天计划做什么，遇到了什么阻塞。前两项对照 subPR 进度核对，第三项由技术负责人和项目经理当场协调。站会之后，项目经理把当日的任务调整同步到 subPR 清单。

5.3 轮值角色与分工

项目采用轮值角色制。一轮中必须有六类责任，但角色不固定到人，按代轮换。

轮值规则有五条：

- 1. 以代为单位轮动。一代需求全部落地并打版本标签后交接，下一代由新的人员接任。

表 5.2 轮值角色权责

轮值角色	开发前	开发中	合并与交接
项目经理	汇总上一代结果；起草下一代 PRD；主持需求评审；确定 subPR 边界与依赖	跟踪需求状态；处理范围冲突；组织必要的需求变更	确认本代需求全部落地；整理版本记录、遗留问题和下一代输入
产品经理	检查用户流程、故事边界和验收标准；防止提前剧透或虚报完成	回答需求问题；检查实现是否偏离玩家体验	按 PRD 做需求级验收；通过后允许合并，不通过则写明差距
技术负责人	确定模块边界、接口和分支策略；识别技术依赖	审查关键实现；协调冲突；保证 main 可运行	拉取最新 main 做技术级测试；检查代码、兼容性、错误反馈和测试证据
模块负责人 / 开发者	领取一个可独立交付的 subPR；确认范围与依赖	在 Feature 分支实现、自测、记录问题；及时同步上游变化	修复评审问题；补齐 PR 说明；完成交接材料
内容与视觉责任人	核对大纲、剧透边界和素材需求；统一画面基调	编写或校对剧情数据，制作和压缩素材，检查文字可读性	核对成品内容与素材授权信息，不让占位素材进入发布版
测试记录人	根据验收条件准备场景、设备和异常数据	记录成功与失败路径，复测修复项	汇总回归结果，确保问题有负责人、有状态、有复现步骤

- 2. **完成即换代。**轮动不等待固定日期。本代提前完成就立即进入下一代需求评审，未通过验收则继续修正，不因周次变化宣布完成。
- 3. **模块在换代时轮动。**成员优先承担与上一代不同的功能域，变更后的负责人写入新一代 PRD 后才生效。
- 4. **作者不能独自验收自己。**subPR 的开发者不能同时是该 PR 唯一的技术审查人或需求验收人。
- 5. **轮换不切断责任。**前任交付 PRD、已合并 subPR、测试记录、遗留问题与最新 main 状态，继任者确认后才开始下一代。

分工按功能域拆分，再按依赖安排并行或串行。功能域分为七块：站点与账

户、剧情与探索、状态与存档、背包与成就、小游戏、内容与视觉、文档与测试。每个模块必须有明确负责人，不出现由全组负责却无人处理的情况。跨模块接口由相关负责人在技术审核日共同确认，例如剧情引擎向探索系统提供节点与调查状态，小游戏把完成结果交回统一状态与存档接口。

5.4 迭代进度

四个版本首尾相接，上一代收尾时下一代已经开始，因此相邻版本的时间区间有重叠。每代结束都产生一个可运行、可演示、可验收的增量。

表 5.3 版本时间轴

版本	时间	核心工作	交付成果
V1	9.3–9.8	需求评审、仓库搭建、目录规范化、页面骨架、基础交互、前置剧情、存档、首个地图小游戏	完整项目架构、13 个页面、可连续游玩的游玩链路、存档体系
V2	9.7–9.11	首轮岗位轮换、PRD 更新、subPRD 拆解、中段剧情、新增小游戏、线索与成就扩展、存档兼容	中段完整剧情、新增交互功能、成就体系、历史存档兼容
V3	9.9–9.15	二次轮换、终局剧情、证据抉择系统、多结局落地、全量测试、文档完善、版本归档	全剧情分支、五个结局、完整成品项目、标准化文档
V4	9.16–9.20	剧情定稿删改、转场效果、过场视频接入、对话框与响应式、小游戏第二阶段、界面细节调整	定稿剧情文本、11 处转场、7 段过场视频、四个小游戏全部完成

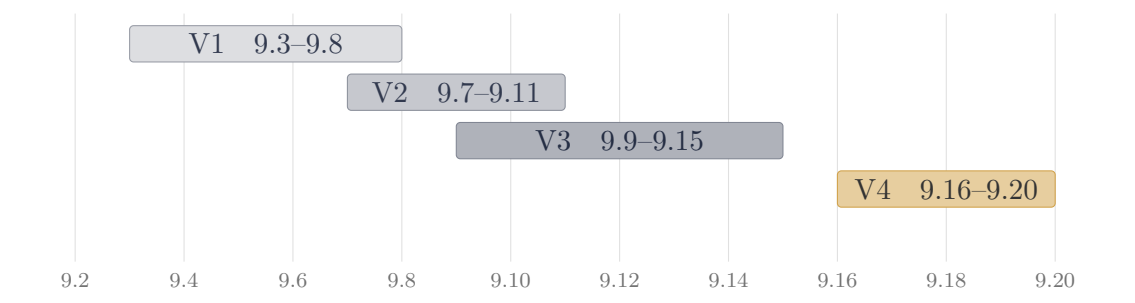


图 5.2 四个版本的迭代时间轴

5.5 流程效果

这套流程在四个版本中保持稳定，效果集中在四点上。

需求驱动。每个版本的起点是需求评审而不是任务分配，subPR 从评审通过的 PRD 里拆出来。需求在评审中完成增删改与合并，并且必须回写到当代 PRD 与 subPR 清单，不留在聊天记录里。复盘日的总结又是下一代的输入，四个版本因此是连着的，不是四段各自为政的工作。

边界清晰。每个 subPR 只解决一个需求，范围在开发前就确认。模块之间通过接口约定通信，改动其中一块不需要通知其他模块。

分工明确。六类责任在每一代都落到具体的人，并且记录在当代 PRD 与 PR 中。轮换之后仍可以查到某一代是谁做的技术审查、谁做的需求验收。

并行友好。各 subPR 在独立分支上并行开发，冲突集中在合入时刻处理，日常开发时间没有被合并工作占用。开发日的每日合并进一步把冲突摊平到每一天，避免积压。

从 9 月 2 日立项到 9 月 20 日，主线累计 229 次提交，共开出 73 个合并请求，其中 68 个已合入。这四个版本的划分与逐日推进的节奏，是这套数据能够形成的原因。

六、测试与回帖情况总结

6.1 浏览器兼容测试



图 6.1 chrome 浏览器



图 6.2 edge 浏览器

6.2 操作系统兼容测试

6.3 分辨率兼容测试

【dddddddddddddddddd
dddddddddddddddddd
dddddddddddddddddd
dddddddddddddddddd
dddddddddddddddddd

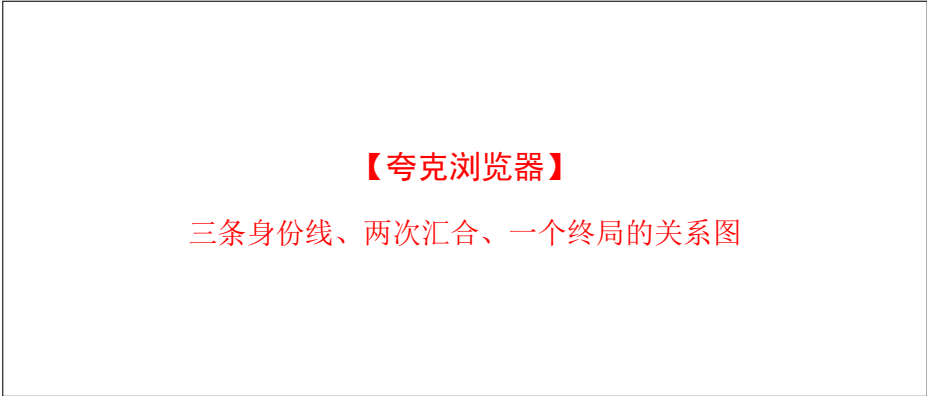


图 6.3 夸克浏览器

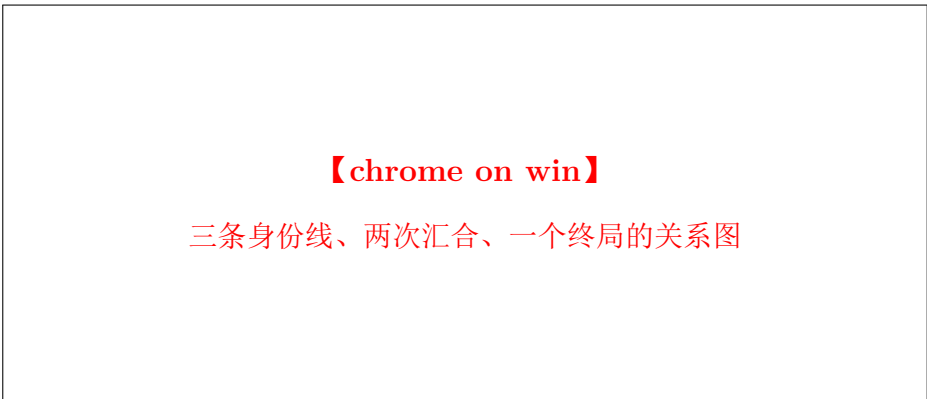


图 6.4 Windows

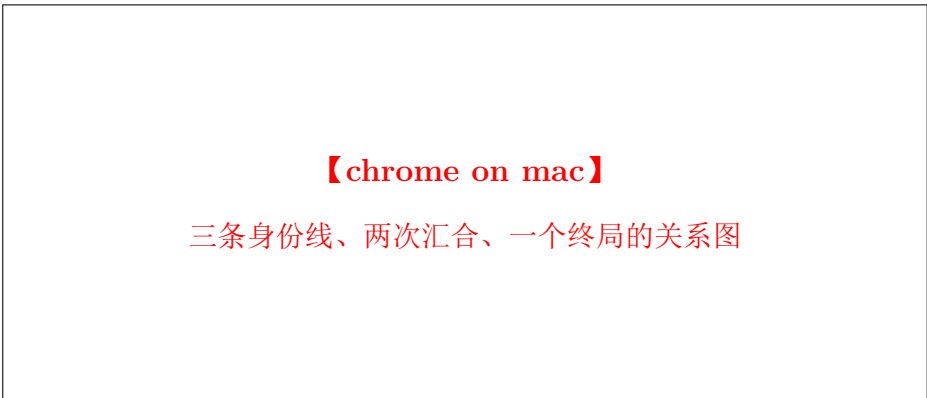


图 6.5 Mac

dddddddddddddddddddddd
dddddddddddddddddddddd
dddddddddddddd】

6.4 回帖情况汇总

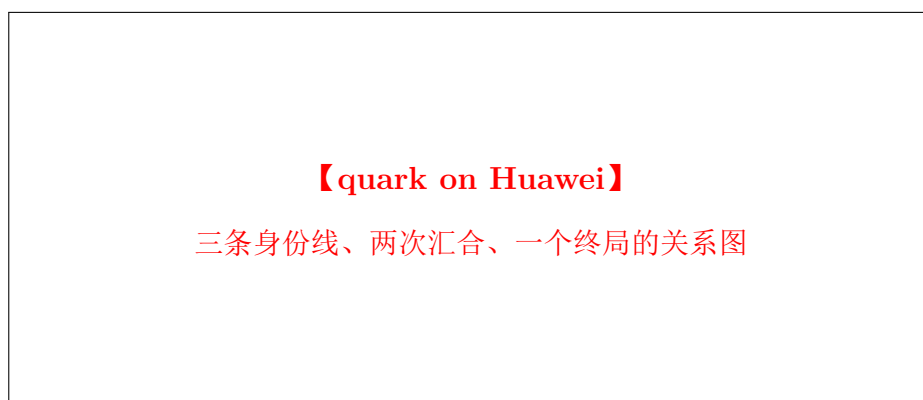


图 6.6 Android

4班-白灯客组-白灯客	4班-白灯客组-白灯客	4班-白灯客组-白灯客
杨 梦 - 2026年09月5日 星期六 14:11		
回顾：4班-白灯客组-白灯客 杨 梦 - 2026年09月5日 星期六 14:12	回顾：4班-白灯客组-白灯客 杨 梦 - 2026年09月5日 星期六 14:14	回顾：4班-白灯客组-白灯客 杨 梦 - 2026年09月5日 星期六 14:31
4班-白灯客组-项目计划	4班-白灯客组-邀请宋河网站	4班-白灯客组-项目日志与会议记录
📎 4班-白灯客组-项目计划.pdf	📎 4班-白灯客组-邀请宋河.rar	📎 4班-白灯客组-项目日志与会议记录.pdf
回顾：4班-白灯客组-白灯客 杨 梦 - 2026年09月6日 星期日 01:23	回顾：4班-白灯客组-白灯客 杨 梦 - 2026年09月6日 星期日 01:23	回顾：4班-白灯客组-白灯客 杨 梦 - 2026年09月6日 星期日 01:23
小组介绍视频	小组介绍视频	小组介绍视频
📎 4班-白灯客组-小组介绍视频.mp4	📎 4班-白灯客组-小组介绍视频.mp4	
回顾：4班-白灯客组-白灯客 杨 梦 - 2026年09月13日 星期日 00:41	回顾：4班-白灯客组-白灯客 杨 梦 - 2026年09月6日 星期日 01:23	回顾：4班-白灯客组-白灯客 杨 梦 - 2026年09月6日 星期日 14:31
4班-白灯客组-项目计划与设计规范	小组介绍视频	git repo 链接：https://github.com/214notfound/jiedeng
📎 4班-白灯客组-项目计划.pdf	📎 4班-白灯客组-小组介绍视频.mp4	📎 4班-白灯客组-游戏原型V1.rar
回顾：4班-白灯客组-白灯客 杨 梦 - 2026年09月13日 星期日 11:25	回顾：4班-白灯客组-白灯客 杨 梦 - 2026年09月6日 星期日 14:31	回顾：4班-白灯客组-白灯客 杨 梦 - 2026年09月9日 星期三 08:11
4班-白灯客组-个人日志套装	git repo 链接：https://github.com/214notfound/jiedeng	今天周二进行集中开发
📎 4班-白灯客组-个人日志（第二周）.pdf	📎 4班-白灯客组-游戏原型V1.rar	
回顾：4班-白灯客组-白灯客 杨 梦 - 2026年09月13日 星期日 11:27	回顾：4班-白灯客组-白灯客 杨 梦 - 2026年09月11日 星期五 17:05	回顾：4班-白灯客组-白灯客 杨 梦 - 2026年09月11日 星期五 17:07
4班-白灯客组-游戏原型与测试视频	电影欣赏网站	4班-白灯客组-中华美食网站
📎 4班-白灯客组-游戏原型与测试视频.rar	📎 4班-白灯客组-电影欣赏网站.rar	📎 4班-白灯客组-中华美食网站.rar
回顾：4班-白灯客组-白灯客 杨 梦 - 2026年09月14日 星期一 13:11	回顾：4班-白灯客组-白灯客 杨 梦 - 2026年09月11日 星期五 17:07	回顾：4班-白灯客组-白灯客 杨 梦 - 2026年09月13日 星期日 00:40
git repo 链接： https://github.com/214notfound/jiedeng	电影欣赏网站	4班-白灯客组-第二周项目日志与会议纪要
回顾：4班-白灯客组-白灯客 杨 梦 - 2026年09月15日 星期二 20:53	4班-白灯客组-中华美食网站	📎 4班-白灯客组-第二周项目日志与会议纪要.pdf
第三周汇报计划与PPT	📎 4班-白灯客组-中华美食网站.rar	
📎 4班-白灯客组-汇报计划.docx		
📎 4班-白灯客组-项目汇报ppt.pptx		
回顾：4班-白灯客组-白灯客 于 知礼 - 2026年09月17日 星期四 15:32	回顾：4班-白灯客组-白灯客 杨 梦 - 2026年09月13日 星期日 00:40	回顾：4班-白灯客组-白灯客 杨 梦 - 2026年09月13日 星期日 00:40
9.16删情v4的pr结束	4班-白灯客组-第二周项目日志与会议纪要	4班-白灯客组-第二周项目日志与会议纪要
	📎 4班-白灯客组-第二周项目日志与会议纪要.pdf	

图 6.7 回帖情况

七、最终显示效果

本章按玩家的实际游玩顺序排列成品截图：从入口页进入，经序章、探索与四个小游戏，走到五个结局，最后是系统页面与多端适配。这 12 张连着看一遍，等于把《借灯》走了一遍。

前面几章分别交代了设计规范、剧情结构与技术实现，本章只呈现跑起来之后的样子。截图均在正式版本上取得，不含原型稿与线框图。

7.1 进入与开场

玩家从入口页进入，经登录或游客通道到主菜单，再走进祠堂，游戏正式开始。这一节的两张图分别是全篇第一印象与第一幕。

入口页是全篇唯一使用毛笔手写字体的位置。冷调雨幕压满整屏，白灯是画面上唯一的暖光源，第三章定下的冷暖对比在这里第一次出现。



图 7.1 入口页首屏

祠堂是序章的全部场景。主角从地上醒来，身上只有三件物品，旁边有一个自称小周的年轻人。这一屏同时是场景层、阅读层与对话框的第一次合流。

7.2 剧情与探索

剧情文本分旁白、独白与角色对白三层，渲染上靠名牌区分：旁白不出名牌，独白出名牌，玩家始终知道一句话是谁说的，或者知道它没有来源。立绘只有左右两个站位，站位变化即表示说话人更换。

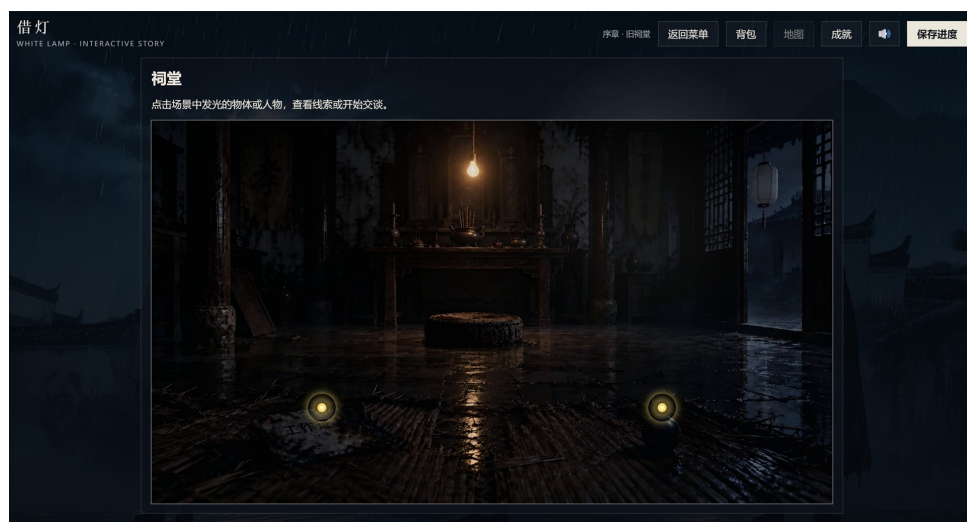


图 7.2 祠堂首屏



图 7.3 人物对白与立绘

探索以整张场景图作交互载体，全篇 18 个场景，覆盖村庄中枢、三条身份线、三处玩法场景和五个结局画面。场景里可调查的对象一律做成热点，调查后留下痕迹或标记为已调查。下面两张是其中的两个地址：村口是唯一的中枢，玩家在这里反复出入，把三条线的进度带回同一个地方；陈家老宅是陈晋年线的第一站，也是全篇第一处门后主光源。

调查得到的物证进入背包，线索独立成页。玩家拿到的是物证，理解的是线索，证据链由线索之间互相印证构成。

7.3 玩法与终局

四个小游戏分别嵌在剧情节点上，玩完把结果回写给剧情模块，由剧情模块决定后续走向。复原手绘地图在村口，玩家从三名村民处取得碎片拼成村庄图；拆解



图 7.4 村口 · 村庄中枢

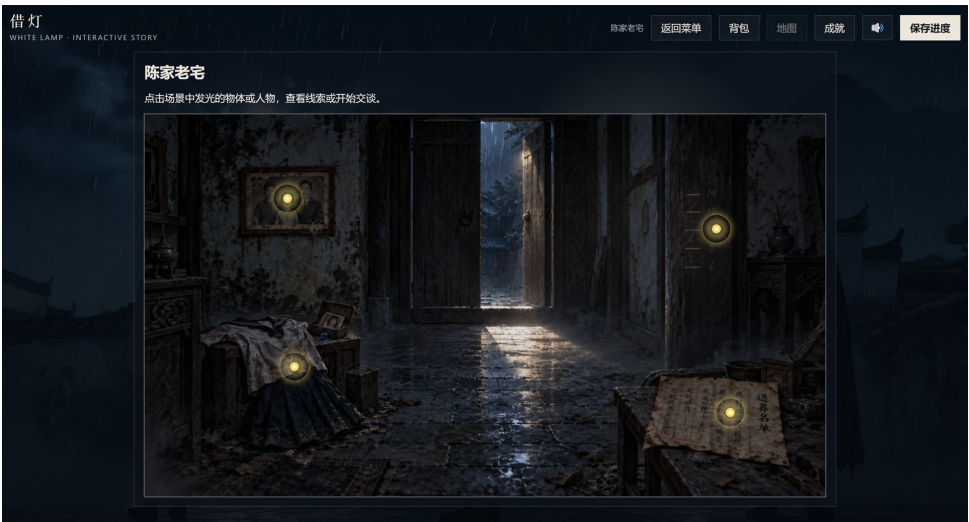


图 7.5 陈家老宅

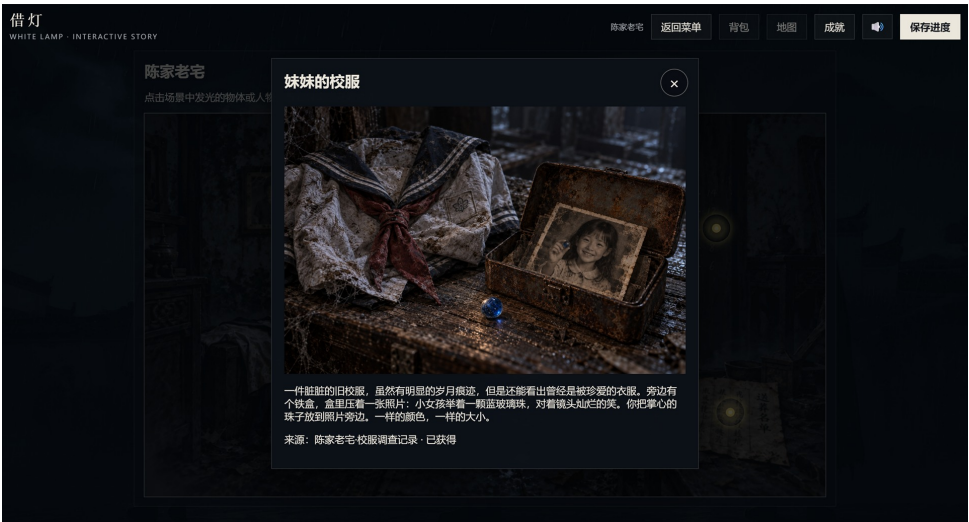


图 7.6 线索详情卡

控制室网络在广播室，逆向追踪童谣、白灯与湿脚印三类人工装置；还原矿井路线在旧矿井，从排水洞绕过封堵进入竖井；逃离数据机房是终局的俯视追逐，躲避小周的追捕、收集三份证据并撤离。下面这张是第四个，它在其中操作要素最全。



图 7.7 逃离数据机房与生命值

终局最后一步不交给对话，而是给玩家一个证据处置界面。三个选项分别处理手里的材料：销毁全部证据、以白灯客的名义选择性公开、完整公开。加上对峙时的交出证据与追逐失败，全篇共五个稳定结局，每个结局配一段过场视频。



图 7.8 证据处置界面

7.4 系统与视听

主菜单、成就页与成员页在主线之外，玩家中途可以随时返回。成就页收录 14 条成就，按剧情推进分成调查札记、灯下真相、终局余烬三组；五条结局成就被标

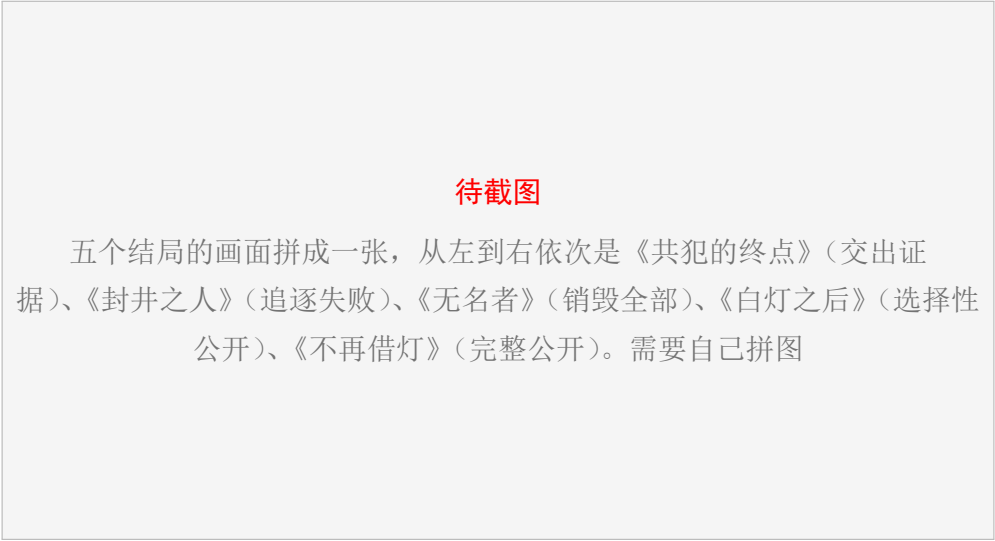


图 7.9 五个结局

记为隐藏条目，未解锁时只显示未明之章配合罗马数字序号，不渲染描述文案。

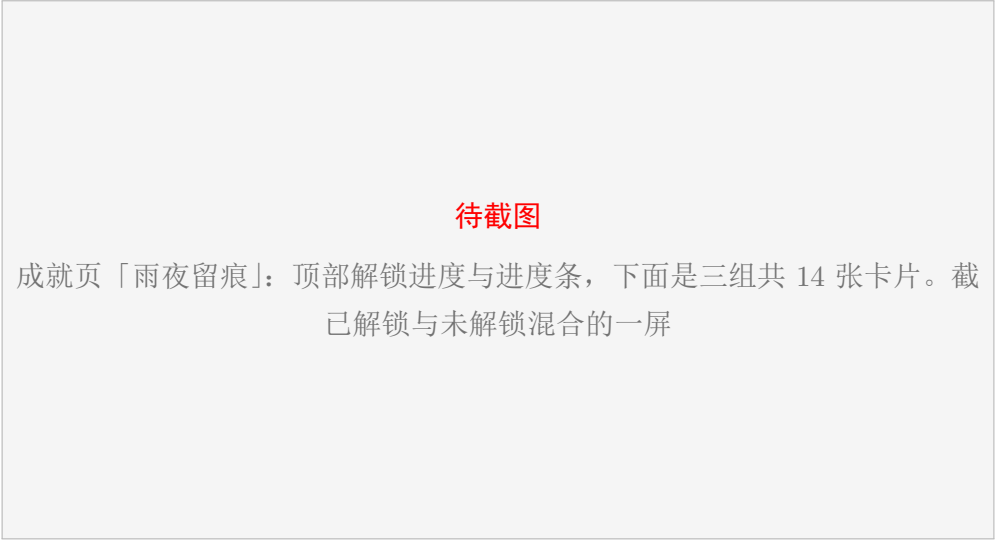


图 7.10 成就页

全站按窄屏重排，热点与阅读框在 390 像素宽下依然可点可读。

转场是全篇唯一使用高强度效果的位置，共十一处，分五种手法：轻闪黑、窒息式闪黑、暴力抖动、无信号花屏、抖动接黑。日常动效压到几乎不可察觉，效果强度集中在这里。

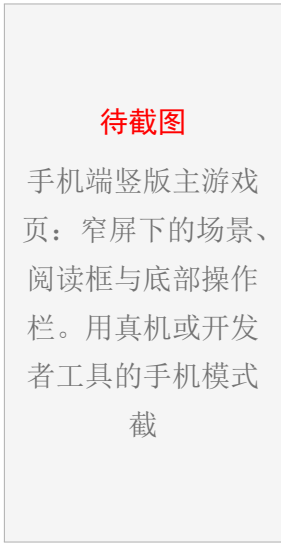


图 7.11 移动端主游戏页



图 7.12 五种转场手法

八、小组总结

9月2日，五名成员各自提出选题方向，逐一陈述并互相质询，当天确定最终方案，完成仓库初始化。9月3日确定开发方式为需求驱动的 subPR 闭环。此后四个版本按同一条链路推进，中途未更换做法。

从立项到9月20日交付，代码规模从空目录增长到92个 JavaScript 文件、15,677行，12个样式表、3,802行，以及35个测试文件、6,122行。本章按版本复盘开发过程，归纳期间反复出现的技术问题与协作机制，并整理团队在方法和判断上的变化。

8.1 项目开发全景复盘

8.1.1 V1：链路打通（9.3–9.8）

V1 的目标是打通一条完整的游玩链路，使玩家可以从封面走到第一周的结尾。

分工按模块划分。于知让负责账户与剧情，罗晨菲负责全局与存档，高冰轩负责小游戏，卢正松负责探索与成就，杨梦负责入口页与介绍页。本代交付13个页面、一套存档体系、前置剧情，以及第一个地图小游戏。

第一版在浏览器上跑通后，提交记录留下两条问题：可玩性不高，文字与背景颜色对比度不足；各板块已经串联，但浏览器端操作路径繁琐。这两条构成 V2 的改进输入。

V1 另有两项事项影响了后续版本。状态模块最初记录了剧情引擎已经拒绝的事件，该问题在 V3 被写成接口契约；团队接口约定草案在本代末被整体废弃，改为按模块拆分的多份文档。

8.1.2 V2：模块边界成文（9.7–9.11）

V2 的主要变化在分工方式。文档从「谁做什么」改为「谁拥有哪些文件、不负责什么」。

五名成员各承担一个模块。罗晨菲负责全局状态与正式集成，高冰轩负责游戏页面与统一阅读组件，于知让负责剧情展示与剧情回归，卢正松负责点击式探索、NPC 对话、背包与成就，杨梦负责场景美术与视觉验收。

每个模块章节的开头都列有「不修改」的范围。罗晨菲不编写剧情内容、热点数据、对话内容、拼图规则和场景美术；卢正松不修改剧情 Node、全局状态、正式存档和 `game.html` 结构；杨梦不修改热点坐标、业务状态、剧情内容和页面切

换代码。

本代交付中段剧情、新增小游戏、线索与成就扩展，以及历史存档兼容。边界成文之后，跨模块问题有了固定的归属对象，该机制的运行情况见 8.3.3。

8.1.3 V3：接口冻结（9.9–9.15）

V3 是三周中记录最完整的一代，产出 28 个剧情 Node 与五个结局。接口契约列出探索、对话、小游戏三类共 14 条交接指令，每条注明发送方、接收方与携带字段。

本代分工收缩为三个方向。高冰轩负责游戏页面与全局状态，卢正松负责点击式探索与整体功能集成，杨梦负责场景美术与剧情回归。于知让承担本代需求文档的编写，罗晨菲完成全局模块的对接；此后两人同期参加数学建模竞赛，未列入本代的模块分工表。

8.1.4 V4：按交付物分工（9.16–9.20）

V4 的分工不再按模块划分，改按交付物划分。

表 8.1 V4 的人员分工

成员	负责内容
卢正松	对话框设计；物品详情卡片与地图碎片的图片替换；场景图片细节微调
杨梦	页面切换转场素材；剧情视频（亮灯、结局）素材与插入；音频素材与插入
高冰轩	三个小游戏设计
于知让	剧情完善；旁白、独白与 NPC 对话改写
罗晨菲	全局状态协调；素材、视频与音效的接入

本代交付 11 处转场、7 段过场视频、四个小游戏、5 段背景音乐与 7 处音效，46 处剧情修改全部落地。

本代出现一次可避免的返工。视频先行完成，随后发现祠堂与结局的场景图需要重画才能与视频衔接，场景图因此返工。

8.2 技术难点与解决

8.2.1 状态记录的信任边界

V1 时期的提交记录写明了问题：状态模块会记录剧情引擎已经拒绝的事件。该问题的后果是错误事实进入存档，此后所有读取存档做出的判断都以错误事实为基础。

V3 将其固化为规则。OBJECT_INVESTIGATED 是探索模块调查完对象后唯一可发送的事件，携带对象 ID 与结果事实 ID，正式状态事件由剧情模块在 commit 中产生。接口补充文档的表述为：「状态模块只接收剧情运行时生成的正式状态提交，不由探索模块直接写 STORY_FACT_RECORDED」。

V3 实施规格另外区分了简写与正式实现。契约文档中「探索模块直接回报状态事件」属于交接层的简写，正式运行时不得采用。该区分的目的是避免后续接手者依照简写实现代码。

这条规则的作用是把存档中的事实限定为经过剧情引擎校验的事实。

8.2.2 界面切换规则的合并冲突

V2 中期，两位负责人分别编写了一份界面切换规则文档。两份文档各自成立，合并时发现八处冲突，合并提交逐条记录了冲突与决策，其中三条如下。

- **界面的并行显示。**一份规定同一时刻只能显示一个状态，另一份允许阅读与背包同时显示。最终确定为一个当前状态加一个锁定背景。
- **拼图的返回位置。**一方主张固定返回探索界面，另一方主张返回阅读界面。最终确定为返回打开拼图之前的界面，由状态字段记录。
- **状态字段的归属。**确定由控制器记录返回状态，发起方只发送打开详情的请求。

八处冲突在同一次合并中全部定案，终稿标注合并日期，此后所有页面切换以该文档为准。冲突集中在文档合并阶段暴露并解决，未进入开发阶段。

8.2.3 测试通过与正式可玩的差距

V3 有一条链路，28 个剧情 Node 全部通过测试。同期检查报告指出，14 条 V3 业务交接指令尚无一条在正式页面完成实现，并明确「不能把 28 个 Node 测试通过等同于 V3 正式可玩」。

报告列出 B01 至 B10 共十项阻断，逐项记录当前证据与建议负责人，其中三项为：

- B01：公共事实表只登记 V1 事实。
- B04：存档校验的白名单来自该事实表，V3 事实无法写入。
- B06：探索校验将节点版本号硬编码为 1。

报告同时规定关闸顺序，B01、B02、B04、B05、B06、B07、B08 必须先关闭；B03 与 B09 可延后，但必须在指定小游戏开工之前完成。

同一批问题中另有一例。玩家进入外围调查节点并读完介绍后，页面停留在阅读层，四个场景入口均不可见。调查确认探索模块已提供四个入口，断点在全局页面状态。修复方式是在阅读器上增加阅读完成回调，由阅读器在最后一个真实阅读块结束时触发；文档同时注明不得伪造完成，也不得以关闭操作替代阅读完成。

单元测试覆盖的是模块内部逻辑，正式可玩还取决于模块在运行时的接入情况。

8.2.4 存档迁移的处理原则

V3 将某节点版本号由 1 升至 2，节点 ID 不变，语义由游戏终点变为老宅结束、完整调查开始。针对三类旧存档的处理如下。

- 停留在该节点且未提交确认：保留节点 ID，版本号更新为 2。
- 已完成该节点：连同事实与完成记录迁移至下一节点。
- 节点、阶段或事实缺失：返回版本不兼容错误，不做补齐。

第三类处理确定了后续所有迁移的原则，即不做推测性补齐。相关规定还包括旧存档事实不得自动改名重用，正式迁移规则由剧情与存档负责人提供。

8.3 协作机制与认知

8.3.1 模块边界的书面化

V2 之后，每份需求文档的模块章节开头均列出「不修改」范围。V3 接口契约进一步明确 14 条交接指令的发送方、接收方与字段，并附注区分对话内容的归属。

边界书面化的效果是改动一个模块不需要通知其他模块。V3 实施规格另有一条约束：探索模块不得根据按钮文字、场景名称或当前 URL 推测对象 ID。该约束的作用是避免通过间接信号建立隐式依赖。

8.3.2 冲突处理的组合原则

V2 至 V3 期间，仓库中留有三份组合审阅文档，对应三组并行提交的合并请求。第一份审阅文档规定，冲突解决必须组合双方语句，不得整段保留一方。

该规定的必要性有两处实例。

- 某份提交删除的样式表中包含仍然有效的声明，仅保留一侧会导致详情页长内容无法滚动。该提交本身无问题，它删除的是重复声明，合并后才发现其中混有一条有效声明。
- 某条分支为所有新获得物品自动打开详情页。合并后，玩家取得旧钥匙时会弹出物品卡片，中断正在进行的阅读流程。修正方式是将自动打开标记限定在三张地图碎片。

这两处问题在任一分支上单独存在时都不会出现，只在合并时出现。

8.3.3 问题的归属与回退

V2 需求文档确定了问题按归属回退的机制：页面结构、焦点与响应式问题回到高冰轩；剧情文本与动作含义回到于知让；热点、调查、对话、背包与成就规则回到卢正松；场景构图与素材回到杨梦。

V3 将该职责交由卢正松承担，并要求先判断问题属于单个模块还是两个以上模块的交接处。

提交记录中有对应实例，包括「修复卢正松提出的两个 bug」「修改杨梦所说的坐标问题」「修复对话读完无法推进的问题」。三个问题由不同成员报出，由归属责任人修复并复测。

场景审查记录了一例完整的修正过程。陈家老宅的正式门开图片重画后，四个热点的原有坐标不再对应，四项坐标逐一对齐；测试同时锁定四个坐标互不重叠，且均在合法百分比范围内，防止后续回退到旧位置。

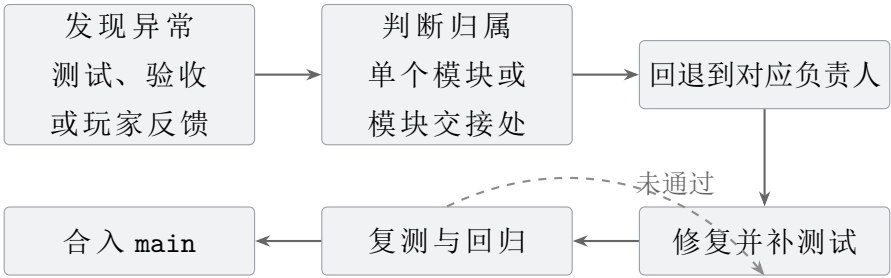


图 8.1 问题归属回退的闭环

8.3.4 文档冲突的处理

三周内废弃两份文档，处理方式一致。

V1 的团队接口约定草案被整体替换，文件头注明该文件已废弃、仅保留用于追溯历史讨论，正文注明剧情模块以哪两份文档为准。

V3 需求文档更换版本，旧版改名为历史版，说明文档标注其仅用于追溯、不作为当前实现依据。新版增加一节，规定 V3 功能开发必须建立在最终 V2 基线之上，不得从尚未包含完整 V2 的旧分支开始批量开发。

两份文档均予保留，保留的目的是记录当时的判断依据。

8.4 项目开发认知

8.4.1 可玩性问题的处理

V1 跑通后的两条问题记录之一，是文字与背景颜色对比度不足。该问题没有通过调整个别色值解决。V2 建立 `tokens.css`，将全站色值与间距收为一份变量表，其余样式表只引用变量名，基调修改一次即可覆盖全站。

体验类问题比功能类问题暴露更晚。自动打开物品详情的改动在功能上没有缺陷，只在玩家实际取得旧钥匙时才会发现问题。

8.4.2 可维护性问题的处理

V4 改写剧情文本时出现三类问题，均不属于代码错误。

- 修改的文件不是实际生效的文件。某 NPC 的显示文本来自较新的数据文件，另一文件中的同名内容早已失效。
- 节点被上层替换，原文件中的两段旁白同样失效。
- 选项的 `description` 字段在归一化阶段被丢弃，写入后玩家不可见。

三类问题的共同点是代码逻辑正确、测试通过，但修改的位置不在生效路径上。剧情数据在 V1 已确定数据与引擎分离的组织方式，实际执行中仍出现了重复数据。

同期完成一项清理，测试中硬编码的块数、条数、末条 ID 全部改为从数据推导。

V4 修复网络小游戏时，提交标题为使其可评审、可诊断。该小游戏此前没有供他人评审与定位问题的入口，本次补充三个评审工具文件与两组交互测试。

8.4.3 验收标准的建立

V3 收尾阶段的进度文档，第一节标题为「尚不能宣称最终交付完成」，列出待完成项：真实浏览器桌面与移动端视觉、键盘焦点、五条结局逐条截图、音频配置。

同一文档的「不得误做」一节列有六条，其中一条为不因素材齐全而宣称功能已经可玩。

发布验收方案将缺陷分为两级。无法进入正式游戏、流程不能推进、存档损坏、资源大面积缺失属于阻断级，须修复后重跑全部门禁；其余属于严重级，须修复后重跑相关链路的全量测试。方案另规定合入后必须从发布分支重新运行门禁，不得复用功能分支上的旧结果。

验收标准在开工阶段即已确定，版本的完成判断由此有据可依。

8.5 总结与不足

从 9 月 2 日立项到 9 月 20 日交付，主线累计 229 次提交，73 个合并请求，其中 68 个已合入。流程中有三项做法保证了推进效率：版本从需求评审开始，不从任务分配开始；模块边界写入文档，不依赖口头约定；问题有明确归属与回退路径。

不足有四项。

一是 V1 的需求文档没有分工记录，五名成员姓名均未出现，第一代的贡献只能通过提交记录回溯。

二是素材类工作的排期。V4 先完成视频，随后发现祠堂与结局的场景图需要重画才能衔接，产生一次可避免的返工。

三是音频交付时间偏晚。V3 收尾时，资源对照表中音频一栏仍为「尚未交付」，跨页面续播的修复完成时也没有配套素材，直到 V4 才接入 5 段背景音乐与 7 处音效。

四是测试断言中一度存在硬编码数值。该问题在同一轮内修复，但反映出编写测试时求快的倾向。

后续版本的改进方向有两项：分工记录从第一代起写全；将相互依赖的素材工作安排在同一阶段完成。

18 天的开发中，五名成员从各自负责独立文件，逐步过渡到在一份接口契约上并行推进。这套协作方式与随之形成的文档体系，是本次项目在成品之外的主要产出。

九、个人总结

本章收录五名成员的个人总结。各节按同一结构组织，分述各人在四个版本中承担的工作、遇到的问题与收获。

9.1 罗晨菲

《借灯》网页交互故事 · 2026 年 9 月 3 日—9 月 19 日

原以为小学期每周只有两节课会很清闲，没有想到需要自己上手一个大型游戏项目的开发。中间还穿插着各种网站小作业、视频剪辑、汇报、项目日志和个人日志。于是我们第小学期硬生生变成了 3 周的紧张刺激的集中开发。与我在开学前参加的黑客松不同，我们的项目人数更多、交接起来更加复杂，游戏项目本身的开发也会比功能相对简单的软件开发要难，而且难点在不同阶段还有不同体现。

9.1.1 V1：全局、导航与存档

在 v1 阶段，我觉得难住我的更多的是陌生技术和团队分工。V1 阶段我就担任了全局、导航和存档的工作。以前只学过 Python、C 语言，要在 3 天时间内，学会 JavaScript 并写出一份合格的全局脚本，一开始我真的觉得不可能。我花了一天的时间快速上手 JavaScript，一边敲代码一边理解它的逻辑；第二天就自己开始按照我的理解写出我的第一个脚本 `state.js`，当时还没确定剧情里面的一部分接口和状态，我就照着 AI 预测的变量和接口写，写出来之后我给 AI 看，它帮我纠正语法、帮我指出没有考虑到的条件和潜在的问题。就这样一遍一遍迭代，最后我才写出适合 v1 的版本。不过，正当我以为我可以写下一个板块的任务时，剧情的同学把接口公布，我才发现我们之间的接口有很多冲突、重叠，又要一个一个变量地修改。到最后我发现，花了大量时间，只完成了一个文件 `state.js`，关于存档 `storage.js` 和导航的 `navigation.js` 还没开动，与此同时我还要制作我们小组的游戏介绍视频... 各种 DDL 吻了上来，我只能一边接住 AI 快速迭代、修改我的代码，一边压缩我的睡眠时间完成任务。这一周无疑是最艰难的。我需要快速了解代码怎么写，全局该干什么，接口怎么统一。不过，也正是第一周的铺垫，我可以在后面几周不紧不慢地完成交付工作，同时哪个部分出问题我都知道该在哪个位置去修改，即便是借助 AI 指导我写 JavaScript，我对我的代码熟悉程度远超过之前 vibecoding 的我。

相关 PR： [文件接口建立 #1](#)、[全局脚本初版 #12](#)、[V1 检验通过 #13](#)。

9.1.2 V2: 需求文档 v2 的制定

在 v2 阶段，最让我感到为难的是需求文档 v2 的制定。第一版文档写的很清楚，但却没有考虑到各板块之间该怎么配合。于是我想在第一版的基础上加上各板块之间该怎么对接，却没想到反而复杂化了。每个人底下都列出需要交接什么交接给谁，最终反而让大家不知道从何入手。需求文档是一个以阶段目标为最终目标倒推我们要干什么事情的文档，追求的是简洁、清晰、易上手。最终我学习到，如果要加上协作部分，就还是应该按照板块拆分，知识在最后加上协作人。这样大家在分别开发的时候就知道该问谁找接口、该怎么联合处理冲突了。制定需求文档的人一定要深度考察当前项目进度，预期每个板块的最终交付是什么，才去动笔写，否则就会很混乱，乱到自己的板块都看不懂要做什么。

相关 PR: 接口交接文档与兼容测试 #17、V2 文档 #18、需求文档 v2 #20、全局模块 #21、界面切换规则合并 #22、全局模块收尾 #24、详情页关闭与玩家反馈 #29。

9.1.3 V3: 备战数学建模

V3 阶段由于我需要集中精力打数学建模比赛，我没有参与这部分的开发，非常感激我剩下的仍在坚守的 3 个队友们，他们各自也有很多事情，深夜还在对接，才让 v4 的时候我们能非常快速地进入到完善阶段。

相关 PR: 本代未提交 PR。V3 期间参加数学建模竞赛，未承担模块开发工作。

9.1.4 V4: 整体完善与成品视频

在 v4 阶段，最难的是各部分的处理和完善。整体框架都搭建好了，但由于前期推进过快，我们还存在一些素材的占位、剧情不够引人入胜，游戏体验不够沉浸等问题。此时我们面对的不是从 0 到 1，而是从 1 到 100 的挑战。我们需要一起完善剧情，完善剧本逻辑和丰满人物形象，我们需要挑选好的游戏素材，插入合适的视频和音效。随时都可能收到队友的反馈，随时都有人 PR，我们需要不断调试，解决冲突，一遍一遍检查游戏有没有漏洞。这虽然没有前两版累，但我们没有放松要求，经常还是需要调试到凌晨。同时，我还是承担了游戏最终产品视频的制作。自己 generate 各种素材，图片、视频、配音，感觉在 AIGC 赛道上也得到了不小提升呢！

相关 PR: V4 全局收尾 #72。

9.1.5 回看这三周

总体来说，这是一段珍贵的经历。我们无人聚在一起，从五个不同的想法，到拧成一股绳一起朝着完美的方向努力，一起把剧本变成一个可视化的游戏。我们付出了很多时间与心血，付出了很多夜晚与白天，但我们收获到的是一段宝贵的开发经验，是如何在大型项目中组织人力，管理时间。感谢赵老师这一路的指导，感谢这门课程带给我们的挑战与启发！

依据：本人撰写的个人总结；文中 V1 至 V4 的成果与仓库中的 PR 合并记录一致。V3 未提交 PR 一节为本人自述，与仓库记录相符。

9.2 杨梦

《借灯》网页交互故事 · 2026 年 9 月 3 日—9 月 19 日

在《借灯》网页悬疑交互游戏项目中，我担任团队 CIO，主要负责美术素材创作，同时承担团队进度管控、会议纪要与项目文档整理等工作。在编程开发方面，我先后完成 V1 版本游戏封面页面开发、V2 版本字体大小与整体配色规范设计、V3 版本剧情旁白与结局内容补充、V4 版本游戏音频与结局转场视频接入工作，全程实操 GitHub 功能分支开发流程，独立完成 PR 提交、参与代码 Merge 合并，完整经历了项目从创意头脑风暴、V1 原型搭建、V2 迭代打磨，到 V3、V4 沉浸感优化、V4 最终版本打磨的整个小学期开发周期。作为中财的学生，我的编程基础比较薄弱，在此之前几乎没有完整的网站开发实战经历，能够从零起步参与网页游戏的设计与落地，这段经历不只是完成课程作业，更是一次全方位的自我突破，无论是技术实操、产品思维，还是团队协作心态，我都收获了很多书本上学不到的感悟。

9.2.1 起步：从组队到能独立提交 PR

回想项目刚刚启动的时候，鼓起勇气迈出组队的第一步，对当时的我来说就是不小的挑战。最初头脑风暴阶段，我也提出过自己的游戏创意方案，虽然最终方案没有被小组采纳，但参与头脑风暴的过程，也让我学会客观看待自己的想法，认真倾听他人创意的闪光点。最开始 GitHub、分支切换、PR、Merge 这些概念对我而言完全陌生，VSCode 环境配置、git 克隆仓库、资源文件目录规范都需要从头学起。我沉下心，跟着团队学习 PRD 驱动的完整游戏开发流程，理解需求评审、模块拆解、任务划分、版本迭代、回归测试、交付验收整套工程化的工作模式。从一开始看不懂提交日志、害怕操作错分支，到能够独立创建功能分支、提交代码改动、发起 PR、配合队友完成 Merge 合并，规避多人协作下的代码冲突。每一步都需要查阅教程、翻看文档，在一次次踩坑当中熟悉团队编码协作的真实流程。

9.2.2 美术创作：画面要服务叙事

在美术创作的工作中，我过去简单地认为美术只需要产出好看的画面，但随着项目推进，我对游戏美术的认知发生了很大改变。我逐步学习悬疑游戏的氛围感营造，不再直接简单套用 AI 生成的图片，而是学会在 AI 图片里面融入自己的设计和思考：参考《纸嫁衣》这类悬疑游戏、悬疑短剧的镜头语言，研究明暗对比、画面节奏、氛围细节，反复修改提示词，对 AI 生成的素材二次调整构图、光影，让画面贴合《借灯》中式山村微恐的故事内核。我完成村口、祠堂、老宅多套场景结构草图与标记草图，绘制主要角色人物形象，并且根据队友的剧情反馈反复迭代人物神态气质。在开发过程中遇到一个很实际的问题：如果祠堂大门直接绘制半开状态，会提前向玩家泄露解谜线索。为了服务游戏叙事逻辑，我专门绘制门开、门关两套独立场景资源。这件事让我深刻意识到，游戏美术不是纯粹的艺术创作，画面必须服务交互、服务剧情、服务玩家的解谜节奏，不能仅凭个人的审美喜好做设计。我还参与撰写项目视觉设计规范，梳理整套项目的字体、色彩、图像使用标准，为小组后续素材制作提供统一参考。

9.2.3 分工之外的工作

项目推进过程中，有大量工作都不在我最初认领的分工之内，很多任务都是我此前完全没有接触过的领域。V1 阶段，我从零学习 HTML、CSS，调试封面页面的布局、特殊字体的加载兼容问题，反复修改，完成游戏入口封面，同时兼顾项目计划书的排版美化。V2 版本，我负责整体字体大小、配色体系的设计，同时作为 CIO 整理每周 DDL 清单，同步小组全部成员；参与游戏多轮测试，记录页面缩放异常、移动端适配 bug，同步反馈给开发同学；产出大量场景、人物、物品详情美术素材。随着项目发展，团队人手紧张，我主动承接了 V3 版本的剧情相关工作。刚开始接触剧情节点体系时，我内心十分忐忑，担心自己无法胜任。我逐份阅读项目文档，研究已有的剧情 node、旁白、结局的编写格式，一点点上手补充旁白文本、完善分支结局逻辑，完成之后提交 PR，真正实现从美术岗位向叙事内容的拓展。进入 V4 阶段，我的工作进一步拓展到音视频素材：我制作五版结局转场视频，为祠堂设计亮灯、灭灯、白灯亮起三种不同状态的场景图，作为视频的首尾帧，反复调试像素尺寸，解决视频和游戏场景错位的问题；同时搜集风声、环境悬疑音效，自学音频接入相关代码，调试参数，把音效整合进游戏流程。很多次工作到深夜，虽然身心疲惫，但每攻克一个全新领域，都能真切体会“边学边做”带来的成长。

相关 PR：入口封面页 #4、素材整理 #48、V3 剧情与结局文本 #51、结局转场效果 #67、音频接入 #74。本节的五项工作与这五个 PR 一一对应。

9.2.4 文档、日志与课堂汇报

作为小组 CIO，除了开发任务，我还要跟进文档、日志、会议纪要的整理提交。我也从中吸取了深刻的教训，曾经因为提交个人日志遗漏截图细节，导致小组作业被扣分。这件事提醒我，一个完整的项目，不只是把功能跑通，文档截图、格式排版、附件完整性这些细碎的细节，同样是交付成果不可分割的一部分，任何小疏忽都有可能影响整个小组的成果。后期我承担课堂汇报 PPT 的修改打磨工作，重新调整截图清晰度、排版布局，撰写汇报讲稿，反复练习，上台完成小组项目汇报。上台之前我十分紧张，手心冒汗，但是完整完成汇报之后，我也克服了公开展示项目成果的心理障碍。课后接收老师的反馈建议，我也会回过头审视我们的游戏，思考如何进一步提升游戏沉浸体验。

9.2.5 不足与回望

当然，我也清晰看到自己身上存在不少短板。面对完全陌生的任务时，我很容易先产生畏难情绪，部分任务前期规划不够充分，会出现临近节点赶工的情况；对于底层业务代码，我大多停留在应用调用层面，深层逻辑理解仍然有限，还有非常多可以提升的空间。

回望整个小学期，从课程最开始的迷茫忐忑，对 web 开发一知半解，到和组员协同，一步步打磨、迭代，最终产出完整可玩的《借灯》网页交互游戏，我的心中有很多感慨。我十分幸运，能够遇到实力强劲又友善包容的队友，在我遇到工具难题、开发卡点的时候，大家愿意耐心对接需求、提供帮助，整个小组互相补位、彼此扶持；我也十分荣幸能够修读这门课程，遇见专业尽责的老师，给我们项目提供宝贵的指导建议。这份从零到一完整做出一款游戏的成就感是难以替代的。这门课带给我的，不只是 GitHub 协作、前端基础、AI 素材处理、音视频制作这些实操技能，更教会我工程化的开发思维，理解跨模块沟通的重要意义，懂得团队项目不能局限在自己的岗位边界，要站在整体产品的角度思考问题。

这段充实又难忘的小学期实践经历，不只是一次课程作业，更是一份宝贵的成长礼物，里面有熬夜调试的辛苦，也有版本迭代完成的喜悦，这些经验都会沉淀下来，成为我未来学习实践路上重要的财富。

依据：本人撰写的个人总结；文中 V1 至 V4 的成果与仓库中的 PR 合并记录一致。

9.3 于知让

《借灯》网页交互故事 · 2026 年 9 月 3 日—9 月 20 日

我在项目中承担两类工作。一类是模块开发，从账户到剧情引擎，再到 V4 的剧情文本定稿；另一类在方法层面，以 PR 驱动需求迭代的开发方式由我在立项

阶段提出。

三周下来，两点体会最深。原创剧情要在一个受限的游戏形式里成立，比写故事本身难。开发方式提出来只是一句话，让它真正跑起来，还需要补齐一整层当时没有意识到的东西。

9.3.1 V1：账户模块、剧情引擎与开发方式

V1 我负责账户与剧情两个模块。账户部分把登录与注册页迁入 assets 目录结构；剧情部分从零实现引擎的第一版，包括节点推进、动作提交与状态记录，并把内容与引擎分开，数据文件存放剧情内容，引擎只负责推进和渲染，不含页面实现。这个划分在 V4 得到验证，46 处文本修订全部落在数据文件上，引擎代码没有改动。

开发方式也是在这一代提出的。9 月 3 日确定以 PR 驱动需求迭代，需求先写成文档，评审确定边界，再拆成若干可独立交付的 subPR 并行开发。提出时的考虑是五个人的工作时间不连续，需要一种能把并行工作隔开的单位。

问题出在执行上。当时我以为把需求讲清楚就足够，没有同时确定文件归属、模块之间的通信字段和每一块的改动边界。结果是 V1 后期团队接口约定草案被整体废弃，改为按模块拆分的多份文档；同期还有一批已经合并的页面改动被整体撤回，重新按接口约定交付。

相关 PR： [账户模块 #2](#)、[剧情引擎与剧情内容板块 #5](#)、[剧情模块微调 #16](#)、[V1 收尾 #19](#)。

9.3.2 V2：剧情接口文档与展示适配

V2 我的模块是 game-line，负责剧情数据与展示适配，工作包括核对 11 个剧情节点的展示类型与等待条件、把阅读完成与操作提交接到统一接口、按三个阶段做剧情回归。这一代的第一个 PR 交付的是剧情板块的接口文档，开发工作开始以接口为起点。

这一代集中暴露的是文档问题。同一件事由不同的人各写一份规则，两份界面切换规则合并时逐条比对，找出八处冲突；我自己交付的剧情接口文档同样经过反复调整。V2 后期团队开始用组合审阅处理并行提交，三组并行 PR 留下三份审阅文档，其中记录的问题大多不是实现错误，而是边界没有提前写明。

相关 PR： [剧情板块接口文档 #23](#)。

9.3.3 V3：需求文档与剧情设定定稿

V3 我承担本代需求文档的编写，内容包括 28 个剧情节点的清单、五条结局的接入顺序与 release-v3 的验收标准；剧情设定也在这一代定稿，五个结局的分支

条件在这一步确定下来。此后我与罗晨菲同期参加数学建模竞赛，没有再承担 V3 的模块主责工作。

V3 的接口契约与 V1 那份被废弃的草案相比，差别在于前者把 14 条交接指令的发送方、接收方与携带字段全部写成了表格，并附注区分哪些写法属于交接层简写、哪些是正式实现。把一件事写清楚需要多少信息量，是 V1 阶段没有估计到的。

相关 PR: [release-v3 合并 #63](#)。

9.3.4 V4: 剧情文本定稿与数据清理

V4 我负责剧情文本的定稿，46 处修改覆盖序幕、中段与结局，包括旁白、独白、NPC 对话的改写和角色签名的统一。

这一轮踩到三个陷阱，都不属于代码错误。某 NPC 的显示文本来自较新的数据文件，另一个文件里的同名内容早已失效；某个剧情节点被上层替换过，原文件里的两段旁白同样不再生效；选项的描述字段在归一化阶段被丢弃，写入后玩家不可见。三处问题的共同点是代码逻辑正确，修改的位置却不在生效路径上。

同期完成一项清理，测试中硬编码的块数、条数、末条 ID 改为从数据推导。剧情文本每调整一次都会牵动断言，写死数字的代价在这一轮集中体现出来。

相关 PR: [剧情与对白修订 #64](#)、[剧情修改第二版 #65](#)、[剧情修改第二版 #71](#)。
#70 与 #71 内容相同，#70 未合并，#71 为最终合入的版本。

9.3.5 回看这三周

原创剧情与游戏形式的关系是这次最费力的部分。

《借灯》的故事线依靠人物的选择链条：陈晋年、王阙、白灯客三个名字下是同一个人，玩家最终要判断的不是相信谁，而是愿不愿意把三个名字下的行为放回同一个人身上。这条线在文本里可以靠叙述控制节奏，进入游戏后多出一层约束，玩家会按自己的顺序点击热点、翻看物品、跳过或重复阅读，任何一处都可能提前泄露后面的信息。

游戏形式本身也限制了剧情能怎么讲。项目是静态站点，没有配音和角色动画，可用的表达手段只有文字、图片、音乐和转场。哪些情绪交给文字，哪些交给画面，哪些留给音乐，需要在写文本的时候就分配好。V4 阶段转场、视频和音频接入之后，同一段文字在不同呈现方式下的效果差别明显，文本也需要跟着调整。

我的结论是，剧情文本从第一稿起就要按可点击、可跳过的阅读方式写，不能按纯文本的思路完成后直接搬进页面。

开发方式这件事的结论也一样。

PR 驱动的迭代方式在四个版本里保持了下来，五个人可以并行推进，冲突集中在合并时刻暴露，而不是积压到最后，这部分是有效的。

执行上的问题是配套没有跟上。提出时只想到要把需求写清楚，没有同时定义文件归属、接口字段和改动边界。V1 的接口草案因此被整份推翻，V2 出现两份界面规则文档合并时的八处冲突，到 V3 才形成可执行的接口契约。现在回头看，需求驱动的方向没有问题，缺的是让需求落地的中间层：需求文档之后必须有接口文档，接口文档必须写清谁发、谁收、带什么字段。这些定完，并行才真正成立。

依据：仓库中的 GitHub PR 合并记录与相关提交，V1–V4 各代需求文档，V3 接口契约与剧情设定定稿。

9.4 高冰轩

《借灯》网页交互故事 · 2026 年 9 月 3 日—9 月 19 日

三周前，我对“完成一个功能”的理解还比较简单：页面能显示、按钮能点击，就觉得任务差不多结束了。随着《借灯》从 V1 推进到 V4，我逐渐发现，玩家看到的是一段连续的故事，开发时却要让剧情、探索、小游戏、素材、页面状态和存档在不同人负责的模块之间准确衔接。对我来说，这三周最大的成长，是从整个游玩流程判断自己的工作是否真正完成。

9.4.1 V1：从单个功能入手，第一次面对团队接口

V1 阶段，我主要制作了项目介绍、团队与成员页面，以及地图拼图小游戏。介绍页让我练习把内容、样式和页面结构组织起来；地图拼图则让我第一次认真处理“玩法规则”和“剧情结果”的边界：小游戏负责判断拼图是否完成，再通过约定的接口把结果交给主流程，而不是自行推进剧情。我还为拼图核心和适配器补了测试，开始意识到交付给队友的并不只是一个能打开的页面。

这一阶段也有一次很直接的教训：早期介绍页的合并需要撤回，再按接口约定调整、重新交付。它让我认识到，个人分支上看起来正确的实现，放进团队的公共页面后仍可能不合适。后来我会先确认文件归属、挂载位置和交接方式，再动手修改。

相关 PR：介绍页初版 #7、调整后的介绍页 #8、地图拼图 #10、介绍页完善 #14、拼图视觉更新 #15。其中 #7 的合并后来被撤回，不能把它当作最终交付成果。

9.4.2 V2：从“把内容放上去”走向“让玩家顺畅地操作”

V2 的工作重心转到正式游戏页。我参与整理页面挂载区域、统一阅读组件、详情卡片、反馈提示和小游戏覆盖层，让剧情文字、NPC 对话、背包与探索信息能在同一页面中有序出现。阅读组件尤其让我体会到交互细节的重量：翻页不应提前提交剧情事实，关闭不能误触发完成；提交失败时，玩家需要看得懂的提示和可用的重试入口。地图入口、图片加载失败时的处理，以及窄屏阅读布局，也需要放到真实流程中验证。

V2 的联调过程并不总是顺利。不同 PR 组合时，样式规则可能互相覆盖，详情内容可能无法滚动，人物对白与旁白也可能挤成一段。我做过修复，也经历过改动被撤回后重新调整。逐渐学会了先复现问题、缩小修改范围，再用页面回归和测试确认没有伤到别人的模块。后期把分散的颜色和尺寸整理为共享样式变量，也是为了让各页面更容易保持一致。

相关 PR：游戏页反馈与资源降级 #28、阅读与页面交互收尾 #34、阅读与详情修复 #36、桌面阅读布局 #39、详情样式整理 #40、对白分段修复 #43、统一页面样式 #45。部分改动经过团队组合审阅后才进入版本。

9.4.3 V3：从负责一块页面，走向负责一条能跑通的链路

V3 扩展了故事场景和结局，我的工作也从局部界面延伸到跨模块联调。我处理了地图拼图在正式流程中的入口和返回，修复游戏页初始化、登录界面与阅读出口的问题，参与接入 V3 正式剧情入口和全局状态。后续联调中，我重点检查探索热点、小游戏命令、结果事实与剧情节点是否能对上，并为重复结果、错误映射和移动端溢出补上约束或测试。背景音乐的播放、静音和跨页面续播，也是这一阶段补齐的体验细节。

这一阶段让我改变了验收标准。一个场景能显示，并不代表玩家能从前一节点进入、完成操作、保存结果，再走到下一节点。只有亲自沿着正式路径走一遍，并检查失败、返回和重复操作，才能判断链路是否可靠。我也开始更主动地读队友的接口文档和 PR，在冲突处讨论责任边界，而不是只守着自己写过的文件。

相关 PR：地图拼图正式流程整合 #49、初始化与登录修复 #52、正式剧情入口 #56、全局状态接入 #57、V3 探索联调 #58、入口与音频修复 #59、移动端布局 #61、发布验收 #62。#58 是团队探索成果的合并，我主要参与其中的命令校验、映射修复与联调。

9.4.4 V4：把三个小游戏从占位入口做成可交付的玩法

V4 是我感受最深的一阶段。此前三个新小游戏有剧情入口和结果交接，但玩家还没有完整的操作过程。我先把需求拆成三种玩法：控制室的线路连通、矿井

的移动探索、机房的证据收集与追逐；再把输入、移动、碰撞、镜头和游戏循环中稳定的部分抽成共享模块，保留各游戏独立的规则。实现后，我将它们接入原有 Gateway，并同步检查剧情提示、角色与场景素材、移动端显示和结果回传。

真正困难的地方在边界情况。线路看似连通却留有断头时不能判成功；追逐中连续碰撞不能让生命值一瞬间归零；玩家取消、技术错误和失败也不能被当成同一种结果。为此，我补了规则与接口测试，并处理退出后的监听器和动画清理。现在我更能理解，玩法“能玩”只是起点，让它能稳定地进入主线、正确地结束和返回，才是交付。9月19日我又继续整理线路谜题的诊断和审阅信息，这部分是 #68 合并后的分支后续改进，不计入该 PR 已合并的内容。

相关 PR： [三套小游戏实现与接入 #68](#)。

9.4.5 回看这三周

《借灯》让我从照着需求完成一个页面，走到能够追问“这个页面为什么这样运行、会不会影响前后流程”。我对 JavaScript 模块、页面状态、事件接口、Git 分支与 PR 协作都有了更具体的认识；也慢慢学会把 AI 辅助生成的代码当作需要自己理解、验证和负责的实现，而不是直接接受的答案。遇到问题时，最有效的办法常常是把现象缩小到一个明确的接口或状态，再和队友一起核对，而不是反复修改表面样式。

我也更喜欢现在的《借灯》：剧情、画面和玩法逐步接在一起后，它不再只是几个人分别完成的任务，而是一段玩家真的可以走进去的 story。仍有视觉细节和操作手感值得继续打磨，但这三周最重要的收获，是我开始具备了把自己的功能放进团队作品中、并为整段体验负责的意识。

依据：仓库中的 GitHub PR 合并记录、相关提交及 V2/V3/V4 开发与验收文档；上文以个人实际改动为主，对团队 PR 明确标注参与范围。

9.5 卢正松

《借灯》网页交互故事 · 2026 年 9 月 3 日—9 月 18 日

9.5.1 V1：认领探索与成就，从头学协作

立项那天我提的是个西游 IP 的方案，想了想开发和设计的难度，自己先撤了。最后大家投票选了《借灯》的剧本，当天晚上一起把剧本和大分工过了一遍。第二天需求评审，我认领了探索和成就两块，同时开始补 HTML、CSS、JS 的底子，边学边写。

说实话那会儿挺懵的。探索模块到底长什么样，我心里根本没画面，就跑去找 GPT 聊，结果我自己给不出像样的约束，它给回来的也全是泛泛的东西，来回好几轮都对不上。这事后面让我吃了点亏，自己没想清楚就去和别人对接，改起来的成本特别高。好在队友带着我，GitHub Desktop 我完全没用过，分支、冲突这些概念都是第一次碰，组长开视频一步一步教我操作，我这才把 PR 的完整流程走了一遍，从建分支、提交代码，到发起 PR、等评审合并。快收尾的时候我把大家第一周的日志收上来汇总了一遍，那是我第一次完整看到另外四个人在干什么。

这一版的收获挺实在。需求评审的作用我现在记得很牢，交付范围不先钉死，后面就是边做边摇摆。还有一条，AI 拿来做头脑风暴可以，但它替不了你自己把事情想清楚，想法模糊的时候应该先画个草图或者找个参考，拿着具体的东西再去和团队谈，比空转快得多。

相关 PR：探索与成就模块 #11。同一功能的早期提交 #3 与 #6 未合并，#11 为最终合入的版本。

9.5.2 V2：先对齐共识，再动手联调

V1 收尾那天的复盘会上，我提了个自己的判断，那些返工的根子不在技术，而在我们从来没就交互怎么做、画面往哪个方向走达成过共识。每个人脑子里一个样，又没什么有效的办法对齐，最后拼到一起风格就是割裂的。

所以 V2 第一件事是找共识。我们定了“光源点击”这个核心交互，借鉴的是恐怖游戏里的处理。定这个东西的时候我学到一个办法，别花半小时争论界面应该是什么感觉，直接甩几张游戏截图或者录屏过去，指着说要的是这种，大家马上就对上了。抽象的话绕来绕去，不如一个具体的参照物。

接下来几天我基本都在做联调。上午和负责界面的同学对接，我把一段真实的 NPC 对话给她，让她拿去做交互设计，这样两边能提前验证接口对不对得上；同时让 Codex 把那份挺笼统的需求文档拆成能执行的清单，我自己也终于想明白每天该推什么、该找谁。下午跑通了热点和坐标的技术链路，定了场景的整体布局图。这一步得跟所有人过一遍，因为每个人的模块都要挂在同一套坐标系上，谁偏一点，别人的交互就错位。那天大部分时间不是在写代码，是在反复确认坐标。

后面还有几件零碎但挺关键的事。逐热点测试的时候，我发现单热点测试过不了、关闭按钮被反馈层挡住，反馈出去修掉了；还有一处布局我是直接截图发给对方让她调的。V2 快收尾时我们还临时改了一次交互，点了光源之后玩家分不清自己是在跟人说话还是在点东西，于是加了一步换景，先切到子场景再对话。改完又得重新定热点、对场景图、跑测试，一整套流程又走了一遍。登录注册页在 VR 场景里的阴影异常、对话框连续重叠这两个 bug 也是这几天修的。中间还帮美术同学用 AI 出了几张物品详情图，她那会儿自己的生成工具用不了，我们顺便把详

情页要展示什么、线索怎么对应、关闭之后回到哪里都讨论定了。

有件事我到现在还记得。收尾那天凌晨，Codex 读完了我给的全部规则和约束，还是跟我说队友那边有东西没给我，让我去要。我印象里“世界树四种存档结果”这个设定是给过的，就没听它的，自己回去翻了一遍，果然早就发过。要是当时真信了，凌晨就去打扰一个已经睡了的同学。AI 对项目上下文的记忆是有边界的，关键的地方得自己拿主意。

相关 PR：探索模块 V2 #27、V2 测试与联调 #30、详情页 U3/U4 #33、换景与坐标对齐 #35、探索详情 #37、背包图片降级 #38、队友 PR 审阅 #41、V1 探索设置清理 #42。单点验证的改动另有一支 #25，未合并。

9.5.3 V3：探索铺开与联调把控

这一段我的日志记得不完整，9 月 13、14 两天是空的，只能按 V2 末那次会上定的职责回述。V3 我接着负责探索部分的铺开，把剧情覆盖从 V2 的局部往完整推；另一块是整体联调的把控，队友的 PR 由我审、再合进主线，保证各模块合入的节奏不断档。美术那边要的十几张场景草图，是我先把对象表格和 ID 清单整理给她，她画完我再反向核对一遍，确认数据传递和画面呈现之间没有偏差。

这段时间最大的变化是我不再只盯着自己的模块了。联调本身就是一条独立的任务线，它的产出不是某段代码，是别人能不能接住我写的这部分。日志缺了两天这件事我也想记一笔，联调最忙的时候我把复盘排到后面去了，等想起来已经补不全，这个毛病我在后面专门写了改进的办法。

相关 PR：V3 场景与热点 #58、坐标问题修复 #60。

9.5.4 V4：对话框、网站与版本收尾

V4 的需求会是从艺术、音效、剧情、沉浸感几个维度拆的，分到我头上的是游戏主题网站和团队日志汇总，另外几件事是过程中接过来的。

对话框是这一版改动最大的一块。剧情那边把原来混在一起的系统对话拆开了，我得跟着重新设计，最后定了旁白、独白、人物对话三种类型，样式和交互都分开，玩家一眼能分清现在读的是哪一种。做完找负责剧情的同学对了下节奏，确认呈现方式和他的推进能对上，然后推到了远端。

换图那部分，物品详情卡片、地图碎片这类线索素材统一换了一轮，细节也顺手微调了。我没坐在电脑前空想，是一边把游戏从头玩到尾一边记，哪里看着别扭、哪里信息不清楚就记下来，回头再改。这样找出来的问题比空想靠谱得多。背包和成就菜单页从 V1 之后就几乎没人动过，这一版由我把它更新到跟当前版本对得上的水平。视频那边我也配合改了场景图，把首帧尾帧的位置留出来，免得视频和静态场景切换的时候太跳。网站这块前期翻了教材，把内容组织和页面结构理了

一遍，又去小红书找了些同类的站参考，想清楚要讲什么再动手，最后做的收尾。

这一段的体会是，项目到了后期，决定成品质量的反而就是这些不起眼的填坑。它们不像写新功能那样有成就感，但坑填不上，东西就跑不顺。

相关 PR：[对话框与响应式 #66](#)、[场景图与详情素材替换 #69](#)。

9.5.5 回看这三周

三周下来，我身上有几处变化比较明显。最开始我关心的是本地能不能跑通，后来更关心玩家能不能懂。V2 加的那一步换景、V4 把对话框分成三种，其实都是在解决同一件事，让玩家不用停下来想自己在操作什么。

以前我觉得自己写好代码就算完事，现在知道一份交付得带上接口约定、坐标和 ID 清单这些东西，让下游能接得住。对 AI 的态度也和一开始不太一样了，GPT 聊想法、Codex 拆文档确实好用，但业务约束准不准、上下文全不全、技术边界在哪，这些只能我自己判断，不能被它带着走。

还有视角上的变化。三次汇总团队日志逼着我每个人在做什么、卡在哪儿看了一遍，反过来也让我把自己那份活在整个项目里重新摆了一遍位置。

下次再做类似的事，有些地方我想改。拿到模糊的任务先出草图或者找参考截图，别直接开写，也别急着问 AI，想清楚了再谈，能省掉很多反复。工程习惯还得补，分支管理、提交粒度、PR 描述、提交前自检这几样我现在做得还是糙。关键节点不能只信一个信息源，AI 说的尤其要拿文档和自己的记忆对一遍。复盘也得和开发同步做，V3 日志缺两天就是教训，越是忙的时候越要留出记录的时间。另外像背包成就页这种沉寂了好几个版本的模块，应该每个迭代扫一次，不该攒到最后统一翻修；换图和场景微调这些活也该有一份统一清单，不然命名乱、缓存不更新、尺寸对不上这类问题会反复冒出来。

要说整体的感受，一个想法最后变成一件能给人玩的东西，中间隔着特别多看不见的环节。接口要约定，坐标要对齐，素材要统一，命名要一致，还得有人愿意在最后几天蹲在那里，把没人愿意碰的细节一点点磨平。

我们组的大家也是真的很热情很认真负责。有人手把手教我用 GitHub Desktop，有人在我做界面适配卡住的时候给反馈，有人在我出不了图的时候替我生成。五个人，四个版本，一条从封面到五个结局的链路，靠的不是谁写得快，是每个人都愿意把边界讲清楚、把短板补上。

遗憾也有，中期复盘抓得不够紧，文档记录的节奏被开发挤掉了不少，下次想早点改掉。剩下的我对自己是满意的，从什么都不会，到会本地写代码和一点一点去了解一个项目如何从 0 到 1，再到动手之前先想一遍别人怎么接，这个变化是我三周里最大的收获。

十、评优申请

《借灯》由五个人在十八天里分四代推出来。这四个版本首尾相接，上一代还在收尾，下一代的需求评审已经开完。七个模块各自生长，同时还要共用一份状态、一张场景图、一套存档格式。项目没有在某一代散架，靠的不是运气或默契。

第四章交代了这个项目由哪些部分组成。本章讲剩下的一半：这些部分之间靠什么咬合，边界靠什么维持，内容变化时为什么不会牵动全局。每一处都给出仓库里的真实代码。

本章引用的代码片段取自仓库中的实际文件。为适应排版，注释译为英文并作了精简，个别长行折行，逻辑未作改动。

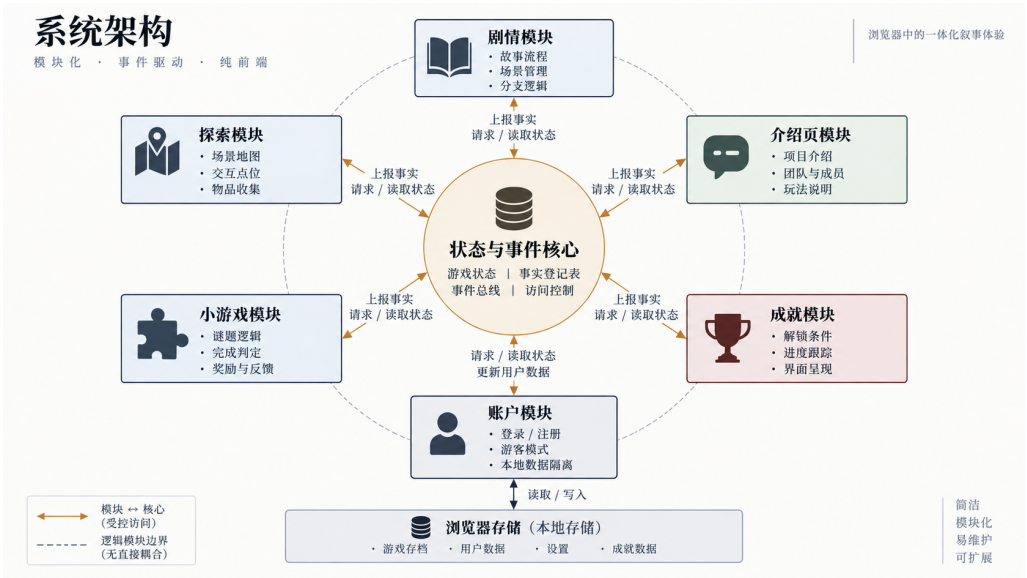


图 10.1 项目技术全景

10.1 模块边界：从文档约定到代码约束

分工写在文档里，文档管不住代码。

V2 时小组对分工文档做过一次调整，从「谁做什么」改成「谁拥有哪些文件、不负责什么」，每个模块的章节开头列出明确的修改禁区。这次调整减少了误会，但没有解决根本问题：文档只能提醒，不能阻止。一个模块仍然可以在代码里写下不属于自己的东西，直到集成那天才被发现。

真正兜住边界的是另一层东西。

游戏进程中所有会改变走向的事情，项目里统一称作**事实**。一个场景被调查

过、一个 NPC 被问完话、一个小游戏完成、玩家拾到某件物品，都是事实。全部事实登记在 `core/game-contract.js` 的一张表里，每条都写明由谁生产。

Listing 10.1 剧情事实登记表（节选）

```
1 // Fact registry. `producer` is the module allowed to report the fact.
2 export const STORY_FACT_DEFINITIONS = Object.freeze([
3   { id: "prologue-wake-context-known", producer: "story" },
4   { id: "white-lamp-witnessed",          producer: "story" },
5   { id: "key-a-given-by-x",              producer: "conversation" },
6   {
7     id: "burned-work-id-investigated",
8     producer: "exploration",
9     externalTargetId: "burned-work-id"
10  },
11  { id: "map-puzzle-completed", producer: "minigame" },
12  {
13    id: "key-a-acquired",
14    producer: "state",
15    derivedFrom: { eventType: "ITEM_ACQUIRED", targetId: "key-a" }
16  }
17 ]);
```

`producer` 有四种取值，对应四类模块。另有一类事实由状态模块自行派生：玩家拾到钥匙这个动作本身就会产生 `key-a-acquired`，不需要任何模块手动上报，`derivedFrom` 字段把这件事交给状态模块自动完成。整张表共 70 条，覆盖全部四个版本。

这张表的价值在执行，不在记录。每条事实都带着归属字段，探索模块报告一条不属于自己的事实时，状态模块不会记下它，而是直接抛错。

Listing 10.2 两处越权检查

```
1 // state.js — the story module may only record facts it owns.
2 const definition = FACT_BY_ID.get(targetId);
3 if (!definition || definition.producer !== "story") {
4   throw new Error(`Story module may not record fact: ${targetId}`);
5 }
6
7 // state.js — external modules are checked against the same registry.
8 for (const factId of event.resultFactIds) {
9   const definition = FACT_BY_ID.get(factId);
```

```
10  if (!definition) {
11      throw new Error(`External event declared unregistered fact: ${factId}
12  });
13  }
14  if (definition.producer !== source) {
15      throw new Error(`Source ${source} may not produce fact: ${factId}`);
16  }
17  }
```

剧情模块想记录一条不属于自己的事实，同样会被拒绝。边界在这里不再是一句约定，而是一个会报错的断言。谁越界，谁在开发当天的自测里就会知道。

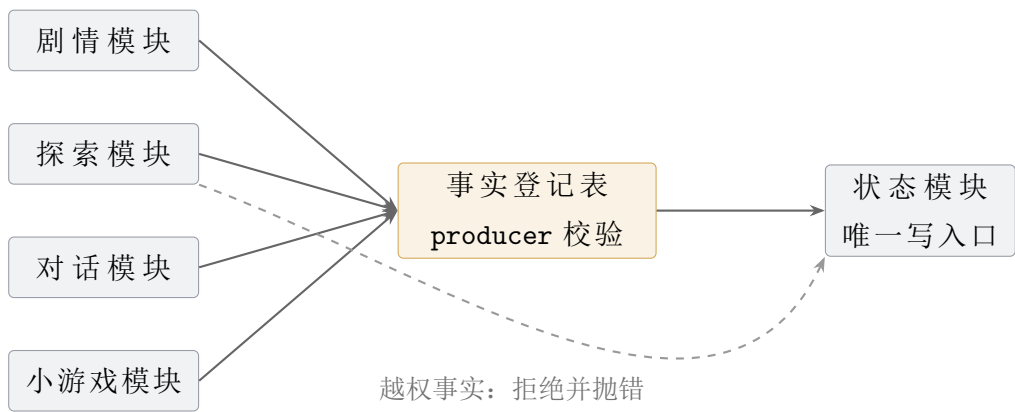


图 10.2 事实的流向与越权拒绝

四条实线是合法路径，模块报告事实、登记表核对归属、状态模块写入。虚线是越权路径，它在登记表这一步就被截断。这张图也说明了一件事：没有任何模块可以直接触到状态，它们只能提出请求，由登记表判断这个请求是否成立。

10.2 状态一致性：唯一写入口与事务提交

游戏的存档是一整棵状态树：读过的剧情、调查过的场景、身上的物品、解锁的地点、成就、小游戏的完成情况。这棵树任何一处写错，玩家都可能卡在某个节点走不下去，或者重复领到同一份奖励。

项目给它设了一条规则：没有任何模块可以就地修改状态。所有更新写在 core/state.js 里，每个入口都返回一个新对象。

外部模块报告事实的那个入口刻着一条刻意的限制，就写在函数上方：它只记录已经发生的事情，不决定下一个剧情节点。探索模块调查完一个场景，它只能说这件事发生了，不能说接下来跳到哪。下一步由剧情引擎根据事实集合自行判断。

这条限制把探索和剧情彻底分开——探索不知道剧情结构，剧情也不关心是谁触发的调查。V1 时这两块由两个人分别负责，整个周期里他们的代码没有互相调用过。

剧情推进走的是另一个入口，也是全项目唯一能改动进度的地方。

Listing 10.3 事务化提交

```
1 // Commit a checkpoint and its state events atomically:
2 // if any step throws, the previous state is left untouched.
3 export function commitStoryTransaction(gameState, requestId, commit) {
4   requireState(gameState);
5   requireText(requestId, "requestId");
6
7   // Idempotent: a request already applied is never applied twice.
8   if (gameState.processedRequestIds.includes(requestId)) {
9     return gameState;
10  }
11  if (!isPlainObject(commit) || !Array.isArray(commit.events)) {
12    throw new TypeError("Story response has no valid commit.events.");
13  }
14  validateCheckpoint(commit.checkpoint);
15
16  // Apply to a draft copy; the old state survives any throw below.
17  let draft = gameState;
18  for (const event of commit.events) {
19    draft = applyStoryStateEvent(draft, event);
20  }
21
22  return {
23    ...draft,
24    ...createStoryCompatibilityProjection(commit.checkpoint),
25    storyCheckpoint: { ...commit.checkpoint },
26    processedRequestIds: addUnique(draft.processedRequestIds, requestId)
27  };
28 }
```

事务提交做三件事：校验整个检查点的结构，在草稿上逐条应用事件，最后一次性替换。任何一条事件抛出异常，草稿被丢弃，旧状态原封不动。玩家不会遇到更新了一半的存档。

重复送达是这类系统里最常见的问题，这个项目同样遇到。同一个请求可能因为快速点击被发两次，同一个小游戏结果可能因为网络或渲染重试被上报两次。项目用三层去重挡住它们：请求 ID、事件 ID、动作的一次性键。三层的判据各不相同，覆盖的重复场景也不同。

有一处处理值得单独说。剧情已经提交成功、但紧接着的渲染或通知失败时，系统不会回滚这次提交。理由是玩家已经看到了内容，把进度退回去等于要求他重玩一遍。这种情况下游戏给出一个存档入口并提示重新进入，而不是假装什么都没发生。



图 10.3 一次剧情推进的完整链路

链条越往下，失败的处理越保守。校验阶段的失败直接拒绝，事务阶段的失败整体丢弃，只有最后一步失败时系统选择保住已经确认的进度。这个顺序是有意的：越靠近玩家的操作越不可逆，系统就越不该替玩家做回退的决定。

10.3 内容演进：数据与引擎的分离

剧情引擎拆成五个文件，每个只做一件事。

最上面一层的文件头注释写明了它的自我约束：只协调请求边界、运行规则和公开响应，不保存任何游戏状态。这句话不是修饰，它解释了为什么同一个引擎能被 V2、V3、V4 三代反复扩展而没有失控——引擎手里没有需要迁移的东西。

引擎在加载时会把全部剧情数据过一遍。每个节点的 ID 引用是否完整、条件字段是否在允许的集合里、跳转出口是否存在、演出条目有没有唯一命中，都在检查范围内。校验不通过，引擎不会带着坏数据跑起来，而是把每一条错误打印出来，并让所有进入剧情的请求返回同一个错误码。

表 10.1 剧情引擎五层的职责与自我约束

文件	负责	明确不做
story-registry.js	装载剧情数据，注册阶段与节点	不校验数据，不推进节点
story-validator.js	启动时检查全部数据引用	不保存状态，不修改数据
story-runtime.js	里程碑对账与效果缓冲	不接触外部模块
story-request.js	请求边界的形态校验	不决定推进结果
story-engine.js	协调请求、规则与响应	不保存任何游戏状态

Listing 10.4 自动循环保护

```
1 // A transition that leads back into an already-visited node would spin
2 // forever. Stop as soon as the count exceeds the number of nodes.
3 transitionCount += 1;
4 if (transitionCount > storyData.nodes.length) {
5   return {
6     node,
7     error: {
8       code: "STORY_INVALID_NODE_DATA",
9       message: "More transitions than nodes: automatic loop suspected",
10      recoveryActions: ["return-menu"],
11    },
12  };
13 }
```

这段代码防的是一种很难排查的错误：数据里的跳转互相指回，游戏在几个节点之间空转，玩家看到的画面停在原地。转移次数超过节点总数就立刻中止，并给出返回主菜单的出口。

V3 需要在不修改 V1 数据文件的前提下追加剧情。项目用了两个办法。追加注册用于新增阶段、事实与小游戏，V1 的文件一个字节不动。替换节点则要求调用方带上这一节点上一版的 revision，对不上就拒绝。两个人在同一代里改同一个节点，不会互相覆盖。

这套结构在 V4 得到验证：46 处剧情文本修订全部落在数据文件上，引擎代码一行没动。改文案的人不需要读懂引擎。

10.4 演出层：不侵入的旁路

11 处转场要在特定台词出现时播放。最直接的做法是在剧情数据里埋标记——某个节点带一个字段，引擎读到就播。

项目没有这么做。

转场写在 `core/transitions.js` 里，是一个独立的观察者。它不修改渲染内核，不修改剧情数据，也不要求任何模块为它提供接口。它只做一件事：监听剧情文本的渲染结果，拿每一段新出现的文本去比对一张表。

Listing 10.5 台词与转场的对照表

```
1 // Line-to-effect table. Reads rendered text only; story data untouched.
2 function has(text, needle) {
3   return text.indexOf(needle) !== -1;
4 }
5 const triggers = [
6   { test: (t) => has(t, "按下播放"), run: () => fx.staticNoise() },
7   { test: (t) => has(t, "一声闷响"), run: () => fx.shake() },
8   { test: (t) => has(t, "一声枪响"), run: () => fx.shakeToBlack() },
9   { test: isDoorCall, run: () => fx.shake() }
10 ];
11
12 // Watch rendered story text and merge one render's DOM changes.
13 new MutationObserver(scheduleScan).observe(story, {
14   childList: true,
15   subtree: true,
16   characterData: true
17 });
```

表中的中文是游戏台词，不是注释。它恰好说明了这个设计的要点：**转场绑在文本上，不绑在节点编号上**。写剧情的人只管写台词，写完不必通知做转场的人去改数据结构；做转场的人也不必理解剧情引擎如何推进节点。两边唯一的约定是一句话本身。

代价同样明确：台词一旦改动，对应的转场会静默失效，不会有任何报错。小组接受这个代价，因为改台词要经过需求评审，而改剧情数据结构的影响面大得多。

扫描被合并进一次微任务。一次渲染会写入多个文本块，DOM 变更可能触发多次回调，直接在回调里扫描会重复执行。项目把同一次渲染的多次变更合并成一次扫描，再用内容 ID 去重，保证一条转场只播一次。

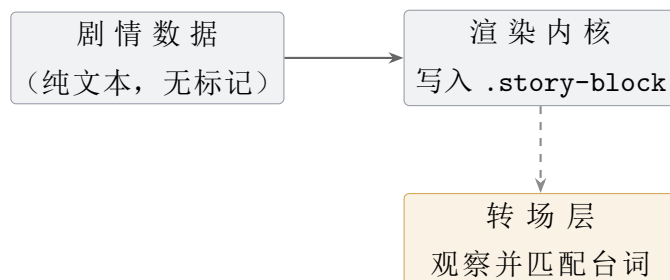


图 10.4 转场层与渲染内核的单向观察关系

虚线只有一条，方向向下。转场层读取渲染结果，不写回任何东西，也不进入剧情数据的链路。删掉这个文件，游戏照常推进，只是少了十一次演出。

10.5 共享内核与跨模块结果交接

四个小游戏里有三个是 V3 新增的，其中矿井地图与终局追逐是实时 2D 玩法。两者的表面差别很大——一个是俯视地图上开关阀门，一个是被人追着跑并收集证据——但底层要处理的是同一批事情：一个稳定的帧循环、相机跟随、碰撞、移动、按键。

项目没有把这两个各写一遍，也没有为此引入游戏引擎，而是抽出七个文件、共 232 行作为共享内核。

Listing 10.6 共享内核的两个片段

```

1 // game-loop.js — fixed step, so a backgrounded tab cannot teleport the
  player.
2 function tick(timestamp) {
3   if (!running) return;
4   const dt = previous
5     ? Math.min(maxDelta, Math.max(0, (timestamp - previous) / 1000))
6     : 0;
7   previous = timestamp;
8   update(dt, timestamp);
9   if (running) frameId = requestFrame(tick);
10 }
11
12 // collision.js — resolve one axis at a time, so corners never leak.
13 export function moveWithCollisions(entity, dx, dy, walls) {
14   const next = {...entity};
15   next.x += dx;
16   for (const wall of walls) {
  
```

```
17     if (!overlaps(next, wall)) continue;
18     next.x = dx > 0 ? wall.x - next.w : wall.x + wall.w;
19 }
20 next.y += dy;
21 for (const wall of walls) {
22     if (!overlaps(next, wall)) continue;
23     next.y = dy > 0 ? wall.y - next.h : wall.y + wall.h;
24 }
25 return next;
26 }
```

步长被夹在 0.05 秒以内。浏览器切到后台再切回来，两帧之间可能隔了几秒，不夹住的话角色会瞬移穿过墙壁。碰撞分两次解算，先横后纵，贴着墙走的时候不会从墙角漏出去。斜向移动用斜边长度归一化，按住两个方向键不会比按一个键快四成。

表 10.2 共享内核被三个小游戏复用的部分

内核文件	职责	矿井地图	管道盘	终局追逐
game-loop.js	固定步长帧循环，防止切后台跳帧	复用	—	复用
camera.js	相机跟随并夹取到地图边界	复用	—	复用
movement.js	归一化对角移动	复用	—	复用
collision.js	分轴 AABB 碰撞解算	—	—	复用
interaction.js	邻近交互判定	复用	—	复用
input.js	键盘输入，失焦时清空按键	复用	—	复用
ui-base.js	入场说明与暂停浮层	复用	复用	复用

内核小到可以一次通读，省掉的重复却不少。矿井地图只需要写阀门与闸门的规则，追逐只需要写敌人行为和证据收集，其余的循环、相机、输入、浮层全部来自共享层。管道盘是网格旋转玩法，用不上实时循环，但它的入场说明与暂停界面同样取自 ui-base.js。

小游戏结束后的结果要交回主线，这一步最容易出脏数据——小游戏可能返回一个剧情不认识的结局，也可能返回两个互相矛盾的结果。项目在交接处做的是集合级校验：检查的不是「返回的是不是一个合法结果」，而是「允许返回的结果集合与契约声明的集合是否完全相等」，多一个少一个都拒绝。

Listing 10.7 小游戏结果的集合级校验

```
1 // The allowed result set must equal the contract's set, not merely
2 // be a subset of it: one extra or one missing result is a failure.
3 const expected = Object.keys(definition.results);
4 if (!Array.isArray(payload.allowedResultFactIds)
5     || payload.allowedResultFactIds.length !== expected.length
6     || new Set(payload.allowedResultFactIds).size !== expected.length
7     || expected.some((factId) => !payload.allowedResultFactIds.includes(
8         factId))) {
9     throw new TypeError("Minigame results do not match the V3 contract.");
10 }
```

这类检查写起来比「判断是否为合法值」麻烦，但它挡住的是真正的风险：剧情按集合相等的前提推导后续走向，只要集合被动过，推导就会出错。把校验做成严格相等，脏数据在交接的那一刻就被拦下，而不是等到几十分钟后的某个结局分支才暴露。

10.6 关键设计取舍

上面每一条做法都有代价。这一节把它们摊开。

表 10.3 四项关键技术取舍

决策	换来的	付出的
不使用框架与构建工具	调试链路最短，克隆即运行	组件复用与页面路由需自建
状态只有一个写入口	模块无法就地写坏状态	入口成为所有模块的关键路径，必须稳定
转场绑台词而非节点编号	剧情与演出互不知情，可并行推进	改台词会使对应转场静默失效
剧情内容与引擎分离	改文案不必理解引擎	数据规模增长，需要校验器

代价不是没算过。立项时小组就知道不用框架意味着组件复用和路由要自己写，也知道转场绑台词会让文案改动带上额外成本。这些代价被接受，是因为它们换来的是另一些东西：交接时不必理解框架的生命周期，改文案时不必理解引擎，做演出时不必理解剧情结构。

三项取舍在后续两代里直接兑现。转场绑台词使 V4 的文本修订与转场调整能够并行推进；内容与引擎分离让 46 处修订没有产生一行引擎改动；唯一写入口把状态相关的问题都收敛到了规则层，第八章记录的那几处返工全部发生在边界

规则的定义上。不用框架的代价兑现得最晚——直到 V4 收尾，自建的组件复用与路由仍然是工作量的一部分——但它换来了最低的启动成本：任何人拿到仓库，不需要安装任何东西就能打开游戏。

10.7 交付完整度

从入口页进入，可以一路玩到五个结局，中间不需要跳过，也不需要打补丁。18 个探索场景、4 个小游戏、5 个结局、11 处转场、7 段过场视频全部接在主线流程里，每一处都能在正常游玩中触发。

`test/` 下有 41 个测试文件、7,295 行。单元测试覆盖存档版本兼容、账户权限、主线与探索的接口契约、小游戏规则与成就判定；浏览器回归测试跑的是真实交互路径，自带本地静态服务器。探索模块的验收用一份按提交哈希冻结的引擎副本，主线怎么演进，这份副本都不变。

仓库里可以查到 229 次提交与 73 个合并请求，其中 68 个合入。四个版本每一代的 PRD、subPR 清单和合并记录都完整保留，谁在哪一代做了技术审查、谁做了需求验收，都能查到。

《借灯》是五个人在十八天里连续滚动推出来的结果，每一代都站在上一代的交付之上，没有哪一代是空转的。作品本身的完整度、技术做法的严谨程度、以及过程中留下的文档与记录，都如实呈现在这份报告里。我们申请对本项目予以评优。

十一、最终得分

《借灯》从 9 月 2 日立项、9 月 20 日交稿，前后十八天。这十八天里，小组在 main 上累计提交 229 次，开出 73 个合并请求，其中 68 个合入。交付的成品包含 18 个探索场景、4 个小游戏、5 个结局、11 处转场效果和 7 段过场视频，中间经历过难以计次大量合作，对接，评测与联调。最终游戏从入口页进入可以一路玩到终局，中间不需要任何跳过，也不需要打补丁，基本达到了最初的项目结项要求。

四个版本是首尾相接的：上一代还在收尾，下一代的需求评审已经开完。这种推进方式决定了没人能只守住自己最初认领的那一块。五个人在四个版本里做过的事几乎都不重样，也几乎都超出了立项当天的分工。基于各自的实际投入，小组一致决定不进行分级评分，申请全员满分。具体评分与理由如下。

11.1 个人评分与理由

11.1.1 罗晨菲

负责全局模块与项目统筹。V1 阶段她是工作量最集中的一个人：仓库的文件接口由她建立，页面骨架、主菜单、导航与存档体系也在这一代从零搭起，9 月 5 日 V1 检验通过时，游戏已经可以连续游玩。V2 阶段转向全局状态与正式集成，交付了项目的全局接口交接文档，并主持了界面切换规则的合并；这一代的 PRD 也由她撰写，模块边界怎么划、subPR 怎么拆，是在这一版定下来的。V3 阶段完成全局模块的对接移交。V4 阶段回到全局状态协调，负责素材、视频与音效的接入收口。开发之外，宣传视频的剪辑和过场视频的接入也由她承担。

得分：100

11.1.2 杨梦

担任团队 CIO，负责美术素材与视觉规范，同时承担进度管控、会议纪要和文档整理。全篇的场景与人物素材出自她手，包括村口、祠堂、老宅三套场景的结构草图与标记草图，以及祠堂亮灯、灭灯、白灯亮起三种状态各自独立的场景图。全站视觉规范里的色板、雨、灯、字体四项由她参与制定，18 个场景能始终保持同一套明暗与色彩，靠的就是这四条。

她的工作范围在四个版本里一再外扩。V1 从零学 HTML 和 CSS，独立完成游戏入口封面页；V2 确定全站的字体与配色体系；V3 在团队人手最紧的时候补上

剧情内容，逐份阅读文档之后接手旁白文本与结局文案，从美术跨到了叙事；V4 一个人做完五个结局的转场视频，又自学音频接入，把 5 段背景音乐与 7 处音效整合进游戏流程。

得分：100

11.1.3 于知让

《借灯》的故事由他创作，9 月 2 日小组投票通过的正是这个剧本。以 PR 驱动需求迭代、subPR 闭环的开发方式也由他在立项当天提出，四个版本都沿这条链路推进，中途没有改过；V1 与 V3 两代的需求文档和模块分工安排由他编写和确定。

模块方面，剧情一直在他手上。V1 从零实现剧情引擎，包括节点推进、动作提交与状态记录，并定下内容与引擎分离的结构。这个结构在 V4 得到验证：46 处文本修订全部落在数据文件上，引擎代码一行没动。V2 负责剧情数据与展示适配，把阅读完成与操作提交接到统一接口。V3 定稿 28 个剧情节点的清单和五个结局的分支条件。V4 完成 46 处剧情文本修订，覆盖序幕、中段到结局，同时涉及大量的 NPC 对话修缮，剧情连贯性推定。

得分：100

11.1.4 高冰轩

全组提交量最多的成员，累计 31 个合并请求。承担小游戏与游戏页两大块，V3 又兼了全局协调与状态存档。V1 制作项目介绍、团队与成员页面，并完成第一个小游戏；V2 集中处理正式游戏页，统一阅读组件、详情卡片、反馈提示与窄屏排版；V3 从局部界面走到跨模块联调，接入正式剧情入口与全局状态，逐条核对探索热点、小游戏命令和剧情节点能不能对上；V4 把三个新小游戏从「只有入口和结果交接」的占位状态，做成玩家真正能操作、能结束、能回到主线的完整玩法。

得分：100

11.1.5 卢正松

负责探索与交互，是贯穿四个版本的联调负责人。V1 承担场景探索、NPC 对话、背包与成就。V2 定下“光源点击”这个核心交互，跑通热点与坐标的技术链路，并把同一套坐标系推给所有人对齐——这一步不做，五个人的模块就挂不到同一张场景图上。V3 把探索从局部铺开到全图，同时负责审查队友的 PR 再合入主线，保证合入节奏不断档。V4 重新设计对话框，把原本混在一起的系统对话拆成

旁白、独白、人物对话三种类型，并完成物品与线索详情卡片、背包与成就页的更新。

得分：100

11.2 关于统一评分的说明

11.2.1 四个版本首尾相接，没有松懈的间歇

四条版本区间是重叠的：V1 是 9.3–9.8，V2 是 9.7–9.11，V3 是 9.9–9.15，V4 是 9.16–9.20。上一代还在收尾，下一代的需求评审已经开完，十八天里没有哪一段是空的。

压力集中在后两代。V3 只有七天，要新增 28 个运行时剧情节点、四个剧情阶段、三个小游戏，还要把五个结局全部跑通；V4 只有五天，要落地 46 处剧情改写、11 处转场效果、7 段过场视频，同时把三个新小游戏补成完整玩法。V3 那段时间，组里几个人是昼夜颠倒过来的，改到凌晨两三点是常态，第二天照常开站会同步进度，工作量完全值得满分。

11.2.2 每个人都身兼数职，且不止于开发

五个人最初认领的都是一个模块，实际做下来的远不止这些。罗晨菲在全球开发之外写需求文档、剪宣传视频；杨梦从美术做到剧情、再做到音频接入；卢正松从探索模块做到全局联调，并且长期承担队友 PR 的审查；高冰轩从介绍页做到小游戏引擎，V3 又接下全局协调与状态存档；于知让从剧情引擎做到两代需求文档和开发方式的制定。全组的会议纪要、每周 DDL 清单、个人日志汇总、课堂汇报材料，也都由成员在开发之外分摊完成。

11.2.3 模块互相咬合，任何一环停下都会卡住整条链路

剧情引擎产出的节点与调查状态要交给探索模块；小游戏判定完的结果要交回统一状态与存档接口，再由剧情模块决定后续走向；场景热点挂在剧情模块给出的对象表格和 ID 清单上，ID 对不上，热点就点不中；转场和视频不绑节点编号，绑在台词文本上，读到约定的句子才播放，改写剧情的人和做转场的人必须对着同一份文本。这四条依赖链上的任何一处停摆，下游的人当天就无法往下做。也正因为这样，没有人能脱离其他人的进度单独完成任务。

11.2.4 工作量有据可查

仓库里的 229 次提交和 73 个合并请求可以逐条查证。四个版本每一代的 PRD、subPR 清单和合并记录都完整保留，谁在哪一代做了技术审查、谁做了需求验收，

都能查到。每一代结束时都产出了一个可运行、可演示、可验收的增量，V4 交付的是从入口页走到五个结局、全流程可玩的完整作品。

综上，《借灯》是五个人在十八天里连续滚动推出来的结果，每一代都站在上一代的交付之上，每个人的工作在整条链路里都无法替换。我们申请不进行分级评分，全员按满分计。望老师批准。